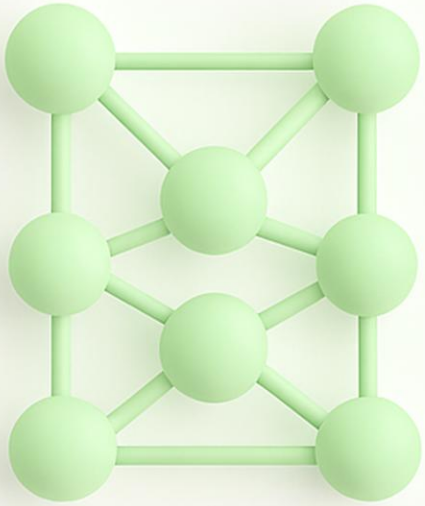


Deep Learning

Lakshmi Narasimhan

IIT Madras

Outline

1. Recap of Machine learning fundamentals
    - Linear algebra and probability
    - Supervised learning: Classification and Regression
    - Loss and errors
    - Model selection: over/under fitting, Bias-Variance tradeoff

Outline

2. Multilayer perceptron
    - Function approximation theorems
    - Feedforward neural networks
    - Activation functions
    - Width, depth and expressivity

Outline

3. Training neural networks

- Backpropagation
- Computational graph
- Optimization: Stochastic gradient descent and variants
- Avoiding overfitting: regularization, dropout, data augmentation
- Gradient instability, skip connection, initializations, early stop
- Normalization: input, layer, batch
- Hyperparameter tuning, curriculum training, transfer learning

Outline

4. Convolutional neural networks
    - Multi-channel inputs
    - Convolution and pooling
    - Padding, striding, flattening
    - Example architectures: AlexNet, GoogleNet, VGGNet, ResNet

Outline

5. Recurrent neural networks
   - Recursive neural networks
   - Backpropagation through time
   - Neural nets with memory: LSTM and GRU
   - Sequence learning and bidirectional LSTM
   - Optimizing RNN: Teacher forcing, leaky units, skipping through time

Outline

6. Attention mechanism
    - Self attention and multi-head attention
    - Query-key-value
    - Attention pooling and scoring
    - Embedding and encoding
    - Transformer architecture

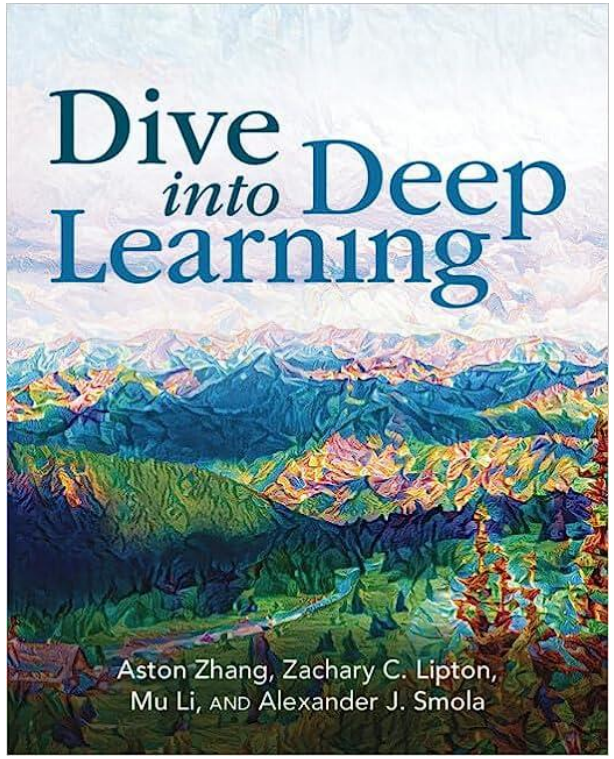
Outline

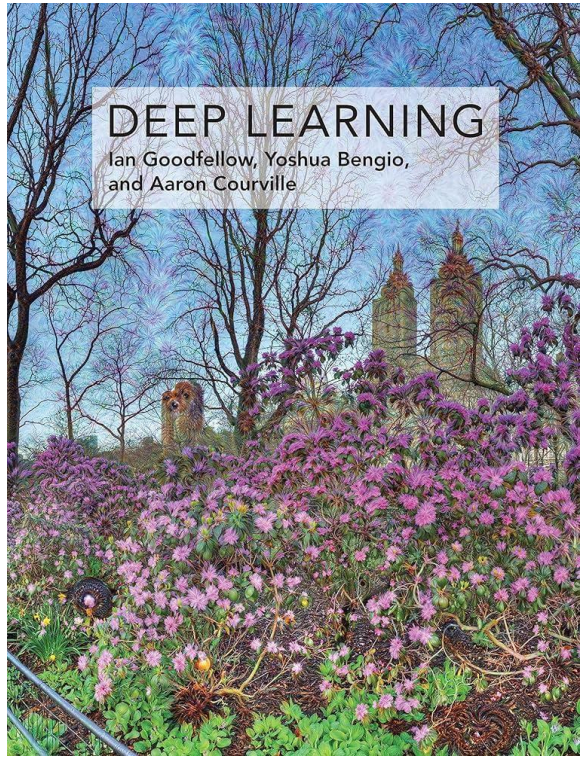
7. Autoencoders
    - Bottleneck architecture for encoder and decoders
    - PCA/sparse decomposition with autoencoders
    - Contractive and denoising autoencoders
    - Variational autoencoder

Outline

8. State-of-the-art
    - Graph neural networks
    - Copresheaf topological neural networks
    - Physics informed neural networks
    - Kolmogorov Arnold neural networks
    - Diffusion and generative models

Reference materials





Logistics

- Lectures: 5.30 PM to 7 PM on Mon & Thu (Jan 5th to Apr 10th)
- Tutorials: On alternate weeks
- Assignments: One per two weeks
  - Theory: 50%
  - Programming: 50%
- Head TAs
  - Manoj Kumar (da24s018@smail.iitm.ac.in)
  - Yuvaram Singh (da24s015@smail.iitm.ac.in)

Grading

- 25% - Mid term, 22 Feb
- 25% - End term, 25 Apr
- 20% - Assignments (4 best out of 6)
- 30% - Project
  - Finalize topic by Feb 28
  - Deadline: April 22
  - Submit: Report + codes

Sample Project Ideas

- Sentence Sentiment Classification
  - https://www.kaggle.com/datasets/jp797498e/twitter-entity-sentiment-analysis
  - https://www.kaggle.com/datasets/kazanova/sentiment140
- Image Super Resolution
  - https://www.kaggle.com/datasets/adityachandrasekhar/image-super-resolution
  - https://www.kaggle.com/datasets/jesucristo/super-resolution-benchmarks
- GenAI with Variational Autoencoders
  - https://www.kaggle.com/code/basu369victor/generate-music-with-variational-autoencoder
  - https://www.kaggle.com/code/averkij/variational-autoencoder-and-faces-generation
- Product Recommender System
  - https://www.kaggle.com/datasets/shivamb/fashion-clothing-products-catalog
  - https://www.kaggle.com/datasets/lokeshparab/amazon-products-dataset

Project Report Format

- Introduction (1 mark)
- Background and state-of-the-art (1 mark)
- Problem Statement (4 marks)
- Dataset Description (2 marks)
- Solution/Methodology/Neural Net Architecture (4 marks)
- Theoretical Analysis (2 marks)
- Data Preprocessing and Training Methodology (3 marks)
- Experimental Results (5 marks for plots & description + 5 marks for attached code)
- Inferences/Insights Obtained/Difficulties faced (3 marks)
- Template - https://www.overleaf.com/read/cbbbwmpqctbv#397b3a

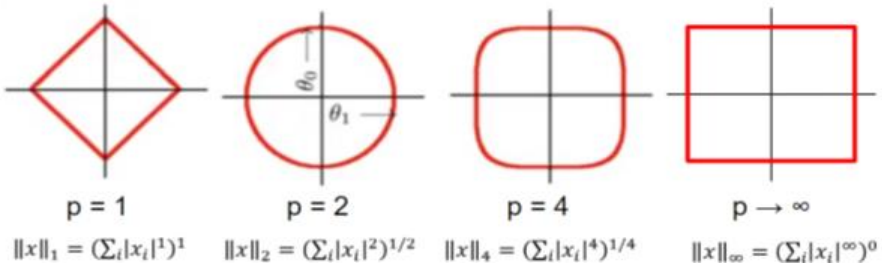
Linear algebra and probability

Vectorspaces

- A set V : $\forall \mathbf{x}, \mathbf{y} \in V$, $a\mathbf{x} + b\mathbf{y} \in V$, where a and b are any scalars
  - Closed under addition and scalar multiplication
- Subspace: a subset of V that is closed under addition and scalar multiplication
- Any element of this vectorspace is a vector
  - For finite-dimensional vectorspaces (e.g., $\mathbb{R}^n$): n x 1 array of numbers
  - In our study, most data samples will be represented by a vector

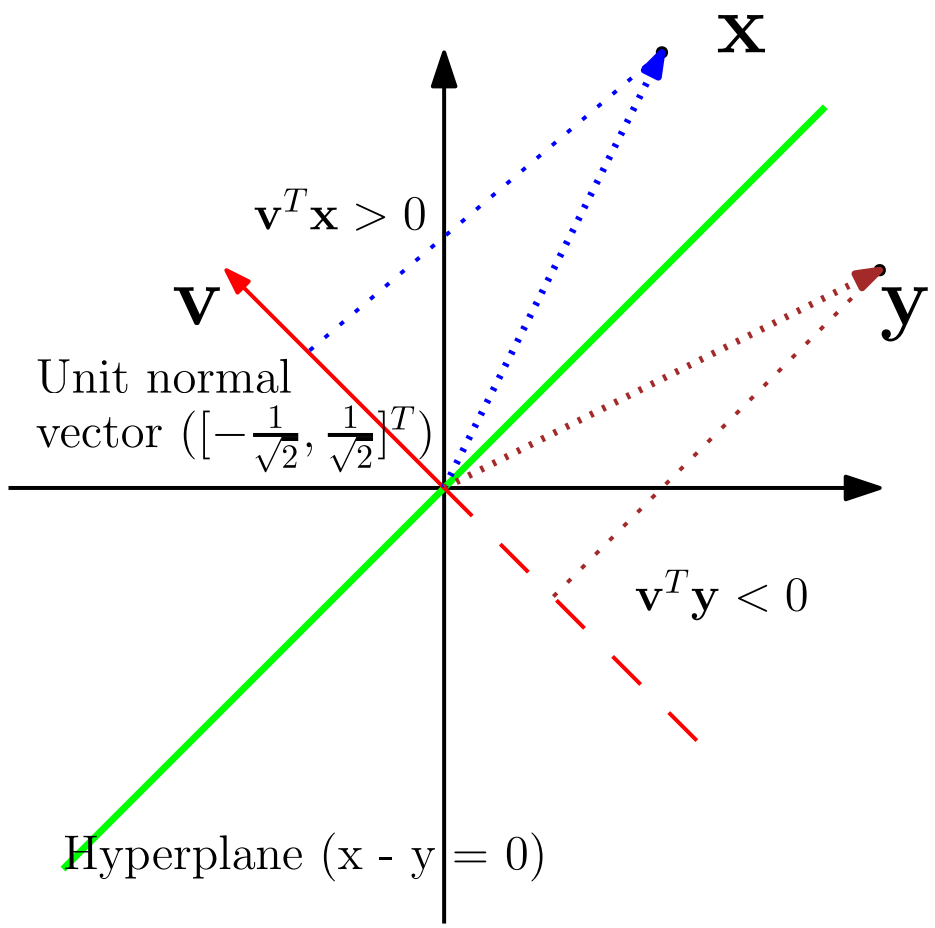
Orthogonality & Norms

- An inner product on $\mathbb{R}^n$: $\mathbf{x}^T\mathbf{y}$
  - Measure of component of $\mathbf{x}$ on $\mathbf{y}$ (and vice versa)
  - Orthogonal: $\mathbf{x}^T\mathbf{y} = 0$ (perpendicular vectors)
  - $\mathbf{x}^T\mathbf{y} > 0$: $\mathbf{x}$ and $\mathbf{y}$ have components along the same direction
  - $\mathbf{x}^T\mathbf{y} < 0$: $\mathbf{x}$ and $\mathbf{y}$ have components along opposite directions
- p - norm ($p \geq 1$) : $\|\mathbf{x}\|_p := \left(\sum_{i=1}^{n} |x_i|^p \right)^{1/p}$
  - Euclidean norm: $\|\mathbf{x}\|^2 = \mathbf{x}^T\mathbf{x}$



Hyperplanes

- Linear hyperplane
  - An *n - 1* dimensional subspace of $\mathbb{R}^n$
  - Passes through the origin
  - Divides $\mathbb{R}^n$ into two halves
  - Represented by a normal vector
- Normal vector:
  - A vector orthogonal to all vectors on the hyperplane
  - Distance of $\mathbf{x}$ from hyperplane = $\mathbf{x}^T\mathbf{v} / \|\mathbf{v}\|$



Singular Value Decomposition

- Singular value decompositon : $\mathbf{M} = \mathbf{U}\Sigma\mathbf{V}^T$
  - Exists for any matrix
  - **U**, **V** are orthogonal: left and right singular vectors
  - Σ is PSD diagonal: The diagonal elements are singular values
  - The diagonal elements of Σ are in ascending order : $\sigma_1 \geq \sigma_2 \geq \ldots \geq \sigma_n \geq 0$
- Pseudoinverse: $\mathbf{V}\Sigma_r^{-1}\mathbf{U}^T$, where r is the rank of **M**
- Matrix norm: $\|A\|_p = \sup_{x \neq 0} \frac{\|Ax\|_p}{\|x\|_p}$ $\qquad \|A\|_2 = \sigma_{\max}(A)$

Random Variable

- Almost all data collected in real life are affected by noise
  - This measurement is an outcome of the data collection experiment
  - The outcome may change every time the experiment is performed
  - However, certain *statistical* properties of these outcomes are deterministic
- Probability space: (Ω, F, P) – quantifying randomness of measurements
  - Ω : Sample space; F : Event space – all possible subsets of Ω
  - P : a measure for all elements of F – e.g., proportion of occurrence of any $u \in F$
- Random variable: a mapping from Ω to $\mathbb{R}$ (real numbers)

PMF vs PDF

Discrete random variables	Continuous random variables
Ω is countable all $w \in \Omega$ has $0 \le P(w) \le 1$	Ω is uncountable $u \in F$ has $0 \le P(u) \le 1$, $w \in \Omega$ has $P(w) = 0$
$X(w)$ is a real-discrete-valued r.v. $P(w)$ is its probability *mass*	$X(w)$ is a real-continuous-valued r.v. $P(w)$ is its probability *density*
Probability mass function (PMF) $P(X = x) = P(w)$ for $X(w) = x$	Probability density function (PDF) $P(X \in [a,b]) = \int_a^b P(x)dx = P(u)$, $u = \{X^{-1}(x), x \in [a,b]\}$
Law of total probability $\Sigma_i P(X = x_i) = P(\Omega) = 1$	Law of total probability $\int P(x)dx = P(\Omega) = 1$

Conditional Probability & Independence

- $P(X \mid Y)$: probability of occurrence of X when Y has occurred
  - The sample space is reduced by conditioning
    - Creates a new probability space and a valid PMF/PDF – $P(X = x \mid Y)$
    - Probability can increase or decrease upon conditioning
  - $P(X \mid Y) = P(X, Y)/P(Y)$
    - $P(X \mid Y = y)$ is a function of x, but $P(X \mid Y)$ is a function of both x and y
- Independent r.v.s: X and Y do not influence each other
  - $P(X, Y) = P(X)\,P(Y) \implies P(X \mid Y) = P(X)$

Bayes Theorem

- $P(X \mid Y) = P(Y \mid X)\, P(X)/P(Y)$
  - Given that Y is observed and a dependent hidden variable X exists, we can infer the probability of the hidden variable
- $P(X)$: Prior, statistics known before any observation
- $P(X \mid Y)$: Posterior, computed indirectly after observing Y
  - $P(X \mid Y = y)$ is a valid PMF/PDF; $P(X \mid Y = y) \propto P(Y = y \mid X)\, P(X)$
- $P(Y \mid X)$: Likelihood, computed based on the outcome of Y
  - $P(Y = y \mid X)$ is not a valid PMF/PDF
    - Note that the probability space itself changes for each value of X

Gaussian Distribution

- Gaussian $X \sim \mathcal{N}(\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{\sigma^2}}$
  - μ, σ : Only two parameters describe the entire PDF
- Multivariate Gaussian: $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{\sqrt{(2\pi)^n \det \Sigma}} e^{-(\mathbf{x}-\mu)^T \Sigma^{-1} (\mathbf{x}-\mu)}$
  - $\mathbf{x} = [X_1, \ldots, X_n]^T$ has n Gaussian r.v.'s that are also jointly Gaussian
  - $\boldsymbol{\mu} = \mathbb{E}\{\mathbf{x}\}$ is the mean vector, $\boldsymbol{\Sigma} = \mathbb{E}\{(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^T\}$ is a PSD covariance matrix
  - If $\mathbf{z} = \boldsymbol{\Sigma}^{-½}\mathbf{x}$, then the elements of $\mathbf{z}$ are independent Gaussian r.v.
  - Precision parameter: $\boldsymbol{\beta} = \boldsymbol{\Sigma}^{-1}$

KL Divergence

- Maximum entropy distributions (priors with least bias)
  - Uniform for discrete r.v.
  - Gaussian for continuous r.v.
- Relative entropy: $D(P \| Q) = \mathbb{E}_P\{\log P/Q\} \geq 0$
  - Measure of how distant is the distribution Q is different from P
- Cross entropy: $H(P; Q) = \mathbb{E}_P\{ - \log Q\} \geq 0$ (for discrete r.v.)
  - $H(P; Q) = D(P \| Q) + H(P)$
  - Minimizing CE is equivalent to minimizing KLD

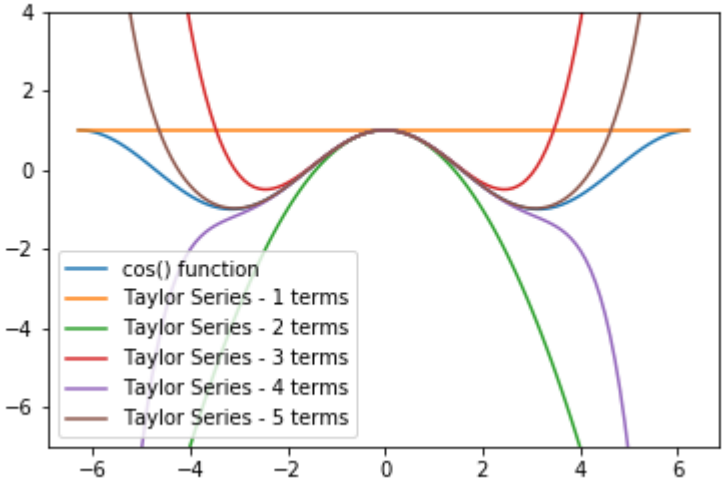
Classical vs Bayesian

Bayesian	Classical
Parameter to be estimated is a random variable	Parameter to be estimated is an unknown deterministic quantity
A prior distribution on the parameters is used	No prior (all values are equally likely - special case of Bayesian)
Posterior probability is optimized	Likelihood is optimized
Poor performance when prior is incorrect	Poor performance whenever the parameter is not uniformly distributed

Taylor Expansion

- Polynomial approximation

$$T(\mathbf{x}) = f(\mathbf{a}) + (\mathbf{x}-\mathbf{a})^{\mathsf{T}} Df(\mathbf{a}) + \frac{1}{2!}(\mathbf{x}-\mathbf{a})^{\mathsf{T}} \left\{D^2 f(\mathbf{a})\right\} (\mathbf{x}-\mathbf{a}) + \cdots$$

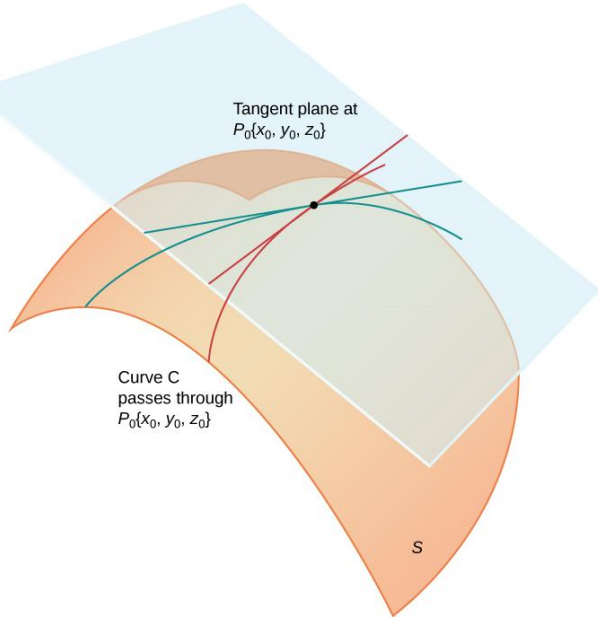


- (

$$\nabla f(p) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(p) \\ \vdots \\ \frac{\partial f}{\partial x_n}(p) \end{bmatrix}$$

essia

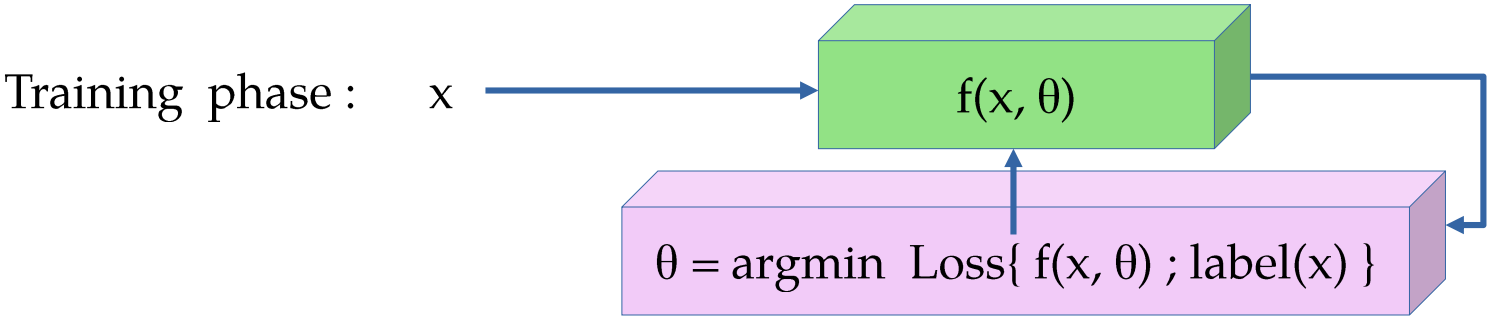
$$\mathbf{H}_f = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1\, \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1\, \partial x_n} \\ \frac{\partial^2 f}{\partial x_2\, \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2\, \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n\, \partial x_1} & \frac{\partial^2 f}{\partial x_n\, \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$



Machine Learning Fundamentals

Supervised/Unsupervised Learning

Supervised Learning	Unsupervised Learning
Training + Testing (inference) phase	Only inference phase
Requires labeled data for training	All data are unlabeled
Examples: Classification & estimation	Clustering, PCA, pattern finding



Loss Functions

- How to find the best parameters while training a supervised learning model?
  - Minimize the error between model output for input x_i and the label of x_i
- Loss function: A quantification of the error obtained during parameter optimization $\theta = \text{argmin}\ \text{Loss}\{ f(x, \theta) ; \text{label}(x) \}$
  - L0 norm (sparsity-promoting), 0-1 loss: $\| y - y_{true} \|_0$
  - L1 norm, absolute error: $\| y - y_{true} \|_1$
  - Squared L2 norm, Euclidean distance: $\| y - y_{true} \|^2_2$
  - Mixed norms: linear combination of different p-norms ($p \geq 1$) – convex losses
  - Cross entropy, F-score, etc.

Regression

- Setup: $y_i = f(x_i) + \varepsilon$
- Find: f(.). Parameterized $f(x) = g(\mathbf{w}, x)$; linear regression: $g(\mathbf{w}, \mathbf{x}) = \mathbf{x}^T\mathbf{W}$
- Given (training data)
  - Labels: $\{y_i\}$
  - Inputs : $\{x_i\}$
- Predict (testing data)
  - Compute {y} for unseen {x}

Classification

- Setup: $x_{i,j} \in C_i$; $i = 1, \ldots, K$ and $j = 1, \ldots, N$
- Find: $f(.)$ s.t. $f(x_{i,j}) = C_i$
- Given (training data)
  - Labels: $\{y_i\}$ s.t. $y_i \in \{C_i\}$
  - Inputs : $\{x_{i,j}\}$
- Predict (testing data)
  - Compute $\{y\}$ for unseen $\{x\}$ s.t. $y \in \{C_i\}$

Classification

- Binary classification: K = 2, C_1 = +1 and C_2 = -1
- Posterior probability for binary classification is

$$\begin{aligned}\Pr(y_i = +1|x_i) &= \frac{\Pr(x_i|y_i = +1)\Pr(y_i = +1)}{\Pr(x_i|y_i = +1)\Pr(y_i = +1) + \Pr(x_i|y_i = -1)\Pr(y_i = -1)} \\ &= \frac{1}{1 + e^{-a}} = \sigma(a); \quad a = \ln \frac{\Pr(x_i|y_i = +1)\Pr(y_i = +1)}{\Pr(x_i|y_i = -1)\Pr(y_i = -1)}\end{aligned}$$

a = log posterior ratios

- The sigmoid function naturally arises here; converts logits to probabilities

Classification

- Posterior probability for K-class classification is

$$\Pr(y_i = C_k|x_i) = \frac{\Pr(x_i|y_i = C_k)\Pr(y_i = C_k)}{\sum_j \Pr(x_i|y_i = C_j)\Pr(y_i = C_j)}$$
$$= \frac{e^{a_k}}{\sum_j e^{a_j}} = \sigma_k(\mathbf{a}); \quad a_k = \ln \Pr(x_i|y_i = C_k)\Pr(y_i = C_k)$$

- The softmax function naturally arises here
- Converts logits (distance from hyperplanes) to probabilities
- Single-layer perceptron with sigmoid/softmax activation

Inference on Gaussian Data

- Binary classification

$$\Pr(\mathbf{x}_i|y_i=1) = \frac{1}{(2\pi)^{M/2}\det(\mathbf{\Sigma})^{1/2}} e^{-\frac{(\mathbf{x}_i-\boldsymbol{\mu}_k)^T\mathbf{\Sigma}^{-1}(\mathbf{x}_i-\boldsymbol{\mu}_k)}{2}}$$

$$\Pr(y_i=+1|\mathbf{x}_i) = \sigma(\mathbf{x}_i^T\mathbf{w}+w_0);\ \mathbf{w}=\mathbf{\Sigma}^{-1}(\boldsymbol{\mu}_1-\boldsymbol{\mu}_{-1})$$

$$w_0 = -\frac{\boldsymbol{\mu}_1^T\mathbf{\Sigma}^{-1}\boldsymbol{\mu}_1}{2} + \frac{\boldsymbol{\mu}_{-1}^T\mathbf{\Sigma}^{-1}\boldsymbol{\mu}_{-1}}{2} + \ln\frac{\Pr(y_i=1)}{\Pr(y_i=-1)}$$

- Multiclass classification

$$\Pr(y_i=+1|\mathbf{x}_i) = \sigma_k(\mathbf{x}_i^T\mathbf{W}+\mathbf{w}_0);\ \mathbf{W}=\mathbf{\Sigma}^{-1}[\boldsymbol{\mu}_1,\cdots,\boldsymbol{\mu}_K]$$

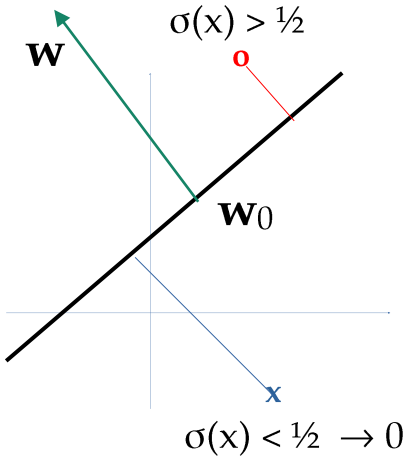
$$\mathbf{w}_0 = [-\frac{\boldsymbol{\mu}_1^T\mathbf{\Sigma}^{-1}\boldsymbol{\mu}_1}{2} + \ln\Pr(y_i=1),\cdots,-\frac{\boldsymbol{\mu}_K^T\mathbf{\Sigma}^{-1}\boldsymbol{\mu}_K}{2} + \ln\Pr(y_i=K)]$$

- Ridge regression

$$\begin{aligned}\hat{\boldsymbol{\theta}} &= \arg\min_{\boldsymbol{\theta}} \|\mathbf{y}-\mathbf{G}\boldsymbol{\theta}\|_2^2 \quad \text{s.t.} \quad \|\boldsymbol{\theta}\|_2^2 \le B \\ &= \arg\min_{\boldsymbol{\theta}} \|\mathbf{y}-\mathbf{G}\boldsymbol{\theta}\|_2^2 + \lambda\|\boldsymbol{\theta}\|_2^2 \\ &= (\lambda I + \mathbf{G}^T\mathbf{G})^{-1}\mathbf{G}^T\mathbf{y}\end{aligned}$$

Classifying Linearly Separable Data

- Hyperplane projection: $(\mathbf{x} - \mathbf{w}_0)^T\mathbf{w} = \mathbf{x}^T\mathbf{w} - \mathbf{w}_0^T\mathbf{w} = \mathbf{x}^T\mathbf{w} - w_0$
  - $\mathbf{w}_0$ is the shift of origin and $\mathbf{w}$ is normal to the hyperplane
- Binary classification
  - When x is on the hyperplane, $\Pr(y = C_1) = \sigma(0) = ½$
  - Farther from the hyperplane $\sigma(\text{ +ve }) \to 1$ or $\sigma(\text{ -ve }) \to 0$
  - Sigmoid output gives the PMF of the Bernoulli r.v. f(x)
- Multi-class classification
  - When x is on the hyperplane, $\Pr(y = C_1) = \sigma_k(\mathbf{0}) = 1/K$
  - Farther from the k^{th} hyperplane $\sigma_k(\text{ +ve }) \to 1$ or $\sigma_k(\text{ -ve }) \to 0$
  - Softmax output gives the PMF of the categorical r.v. f(x)



Logistic Regression – Single Layer Neural Net

- Let the classes be one-hot encoded (to represent a PMF)

$$\Pr(\mathbf{y}|\mathbf{x},\mathbf{W}) = \prod_{k=1}^{K} \Pr(y = C_i|\mathbf{x},\mathbf{W})^{y_k}; \qquad \sum_{k=1}^{K} y_k = 1; \qquad \sum_{k=1}^{K} \Pr(y = C_k|\mathbf{x},\mathbf{W}) = 1$$

- To find $\mathbf{w}$: do maximum likelihood regression

$$\arg\min_{\mathbf{W}} -\ln \Pr(\mathbf{y}|\mathbf{x},\mathbf{W}) = \arg\min_{\mathbf{W}} -\sum_{i=1}^{N}\sum_{k=1}^{K} y_{i,k} \ln \sigma_k(\mathbf{x}_i^T\mathbf{W}) = \arg\min_{\mathbf{W}} \sum_{i} H(y_i; \sigma_k(\mathbf{x}_i^T\mathbf{W}))$$

- The optimal hyperplanes are obtained using gradient descent

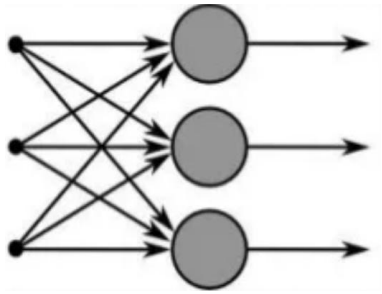
$$\mathbf{w}_{j,t+1} = \mathbf{w}_{j,t} - \eta\nabla_{\mathbf{w}_j}\mathcal{L} = \mathbf{w}_{j,t} - \eta\sum_{i}(\sigma_j(\mathbf{x}_i^T\mathbf{W}_t) - y_{i,j})\mathbf{x_i}$$

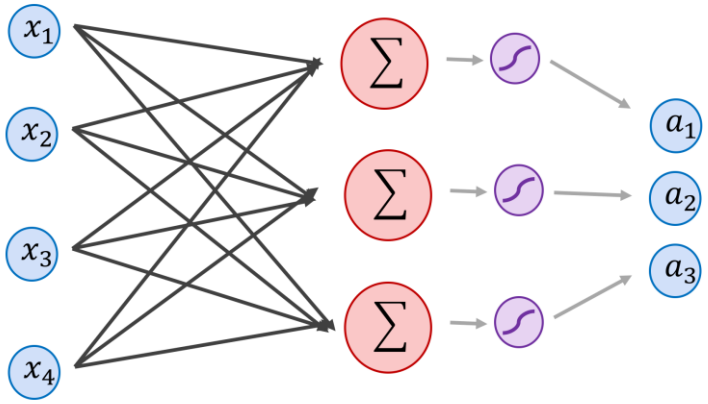
Summary

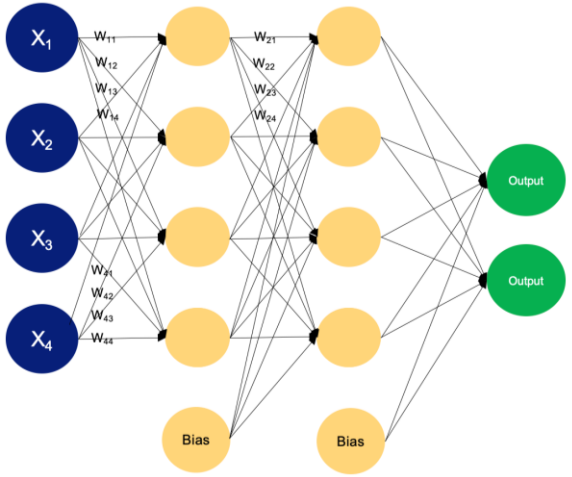
- We focus on parametric supervised learning framework
- Neural models can be viewed as interpolators or curve-fitters
- Loss function: measure of deviation from expected output/label
- Optimal linear/ridge regression: a single layer linear neural net
- Optimal linear classification: a single layer neural net with sigmoid/softmax activation
- Learning weights from training data implicitly learns data prior

Extending to Multiple Layers

- We cannot model XOR with a linear classifier
- What if cascade multiple single layer neural net?
  - Similar to making intermediate decisions
- Can we keep increasing the number of layers/parameters?

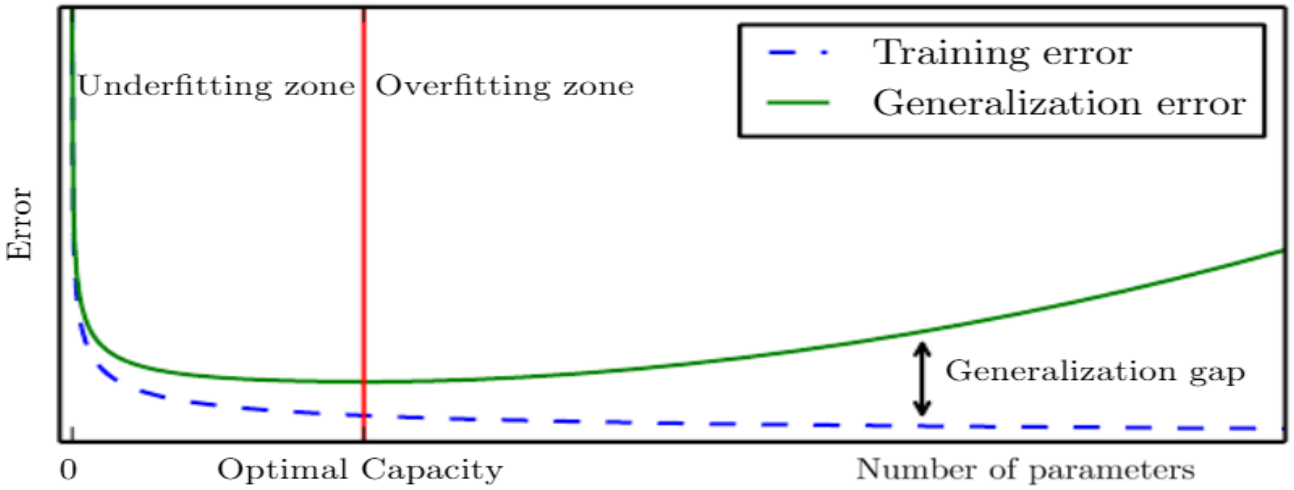


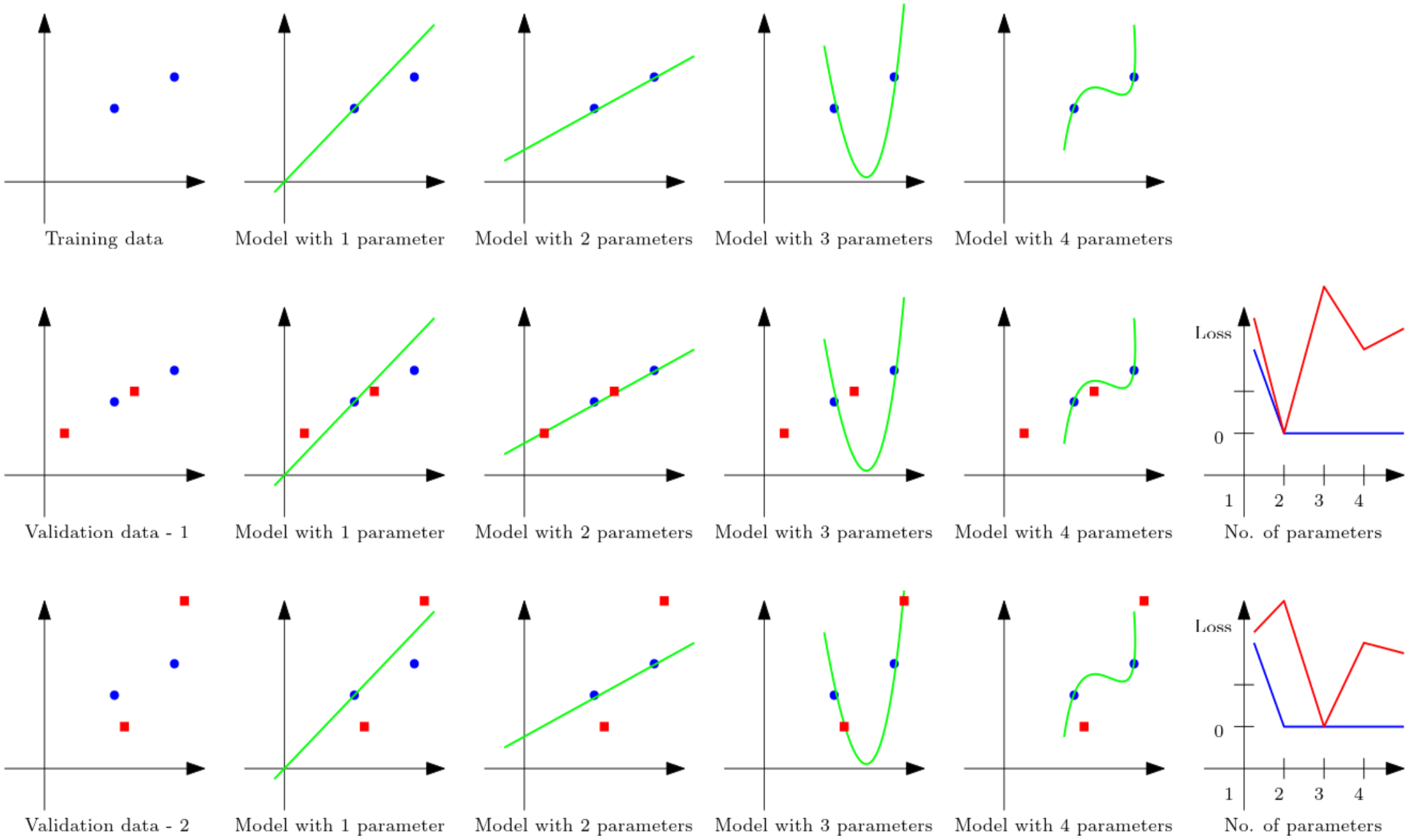




Training/Generalization Error

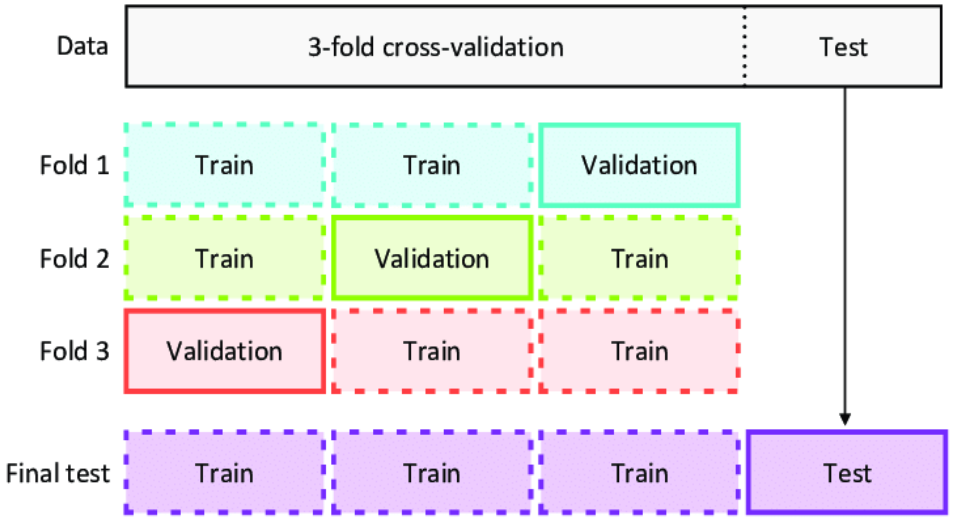
- Increasing number of parameters is not always good
  - Training error may go down, but testing (generalization) error may be high
- There is an optimal number of parameters for a given kind of data
  - VC dimension ($|\text{Test error} - \text{Training error}| \leq O(d_{VC} \log n/n)^{1/2}$)
- Underfitting: Less than optimal
- Overfitting: More than optimal
  - How to identify overfitting?




Training data
Model with 1 parameter
Model with 2 parameters
Model with 3 parameters
Model with 4 parameters
Validation data - 1
Model with 1 parameter
Model with 2 parameters
Model with 3 parameters
Model with 4 parameters
Loss
0
1
2
3
4
No. of parameters
Validation data - 2
Model with 1 parameter
Model with 2 parameters
Model with 3 parameters
Model with 4 parameters
Loss
0
1
2
3
4
No. of parameters

Validation Error & Overfitting

- Split available labeled data to training + validation sets
  - Sequentially minimize both training and validation errors
  - Use validation set to identify overfitting
- Validation error: The error measured with validation set – An indicator of generalization error

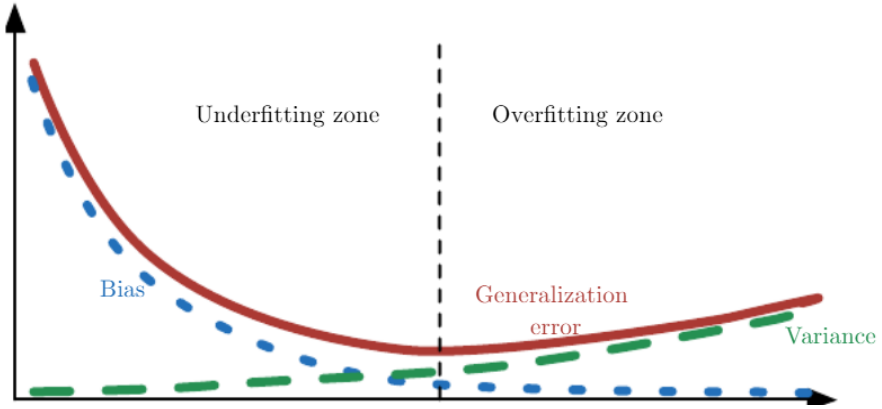


- Cross validation: When very few labeled data is available
  - Use different training+validation split in each iteration of training

Bias-Variance & Model-fitting

- Bias: $\mathrm{E}\{\mathrm{y} - \mathrm{y}_{\mathrm{true}}\}$ – average deviation from true value
- Variance: $\mathrm{Var}\{\mathrm{y}\}$ – average deviation of inference
- Is lower bias or variance always good?

$$\begin{aligned}\mathbb{E}((y-\hat{y})^2) &= \mathbb{E}(y^2) + \mathbb{E}(\hat{y}^2) - 2\mathbb{E}(\hat{y})\mathbb{E}(y) \\ &= \mathbb{E}(y^2) - \mathbb{E}(y)^2 + \mathbb{E}(\hat{y}^2) - \mathbb{E}(\hat{y})^2 - 2\mathbb{E}(\hat{y})\mathbb{E}(y) + \mathbb{E}(y)^2 + \mathbb{E}(\hat{y})^2 \\ &= Var(y) + Var(\hat{y}) + \mathrm{Bias}^2\end{aligned}$$



- Overfitting : High model variance and low MSE in training
- Underfitting : Low model variance and high MSE in training

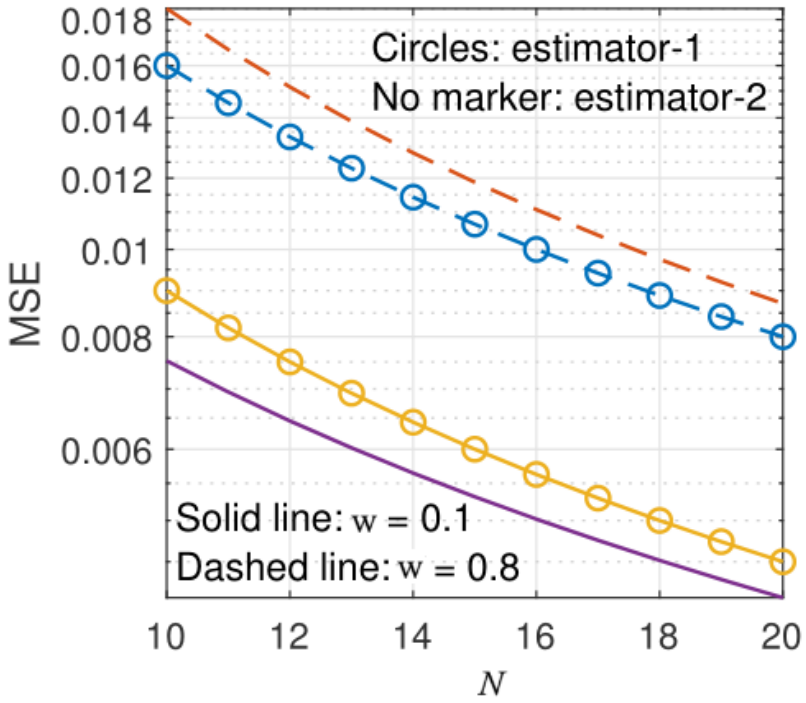
Bayesian Inference

- Infer bias (p) of a coin from observations of its toss: $X_i \sim Be(p)$
- (Classical) No bias

$$\hat{p}_1 = \arg\max_p p^{\sum y_i}(1-p)^{N-\sum y_i} = \frac{\sum y_i}{N}$$

$$Var(\hat{p}_1) = \frac{p(1-p)}{N} \quad \text{Bias}(\hat{p}_1) = 0$$

$$\hat{p}_3 = \arg\max_p p^{\sum y_i}(1-p)^{N-\sum y_i} P(p) = \frac{\sum y_i}{N+1}$$



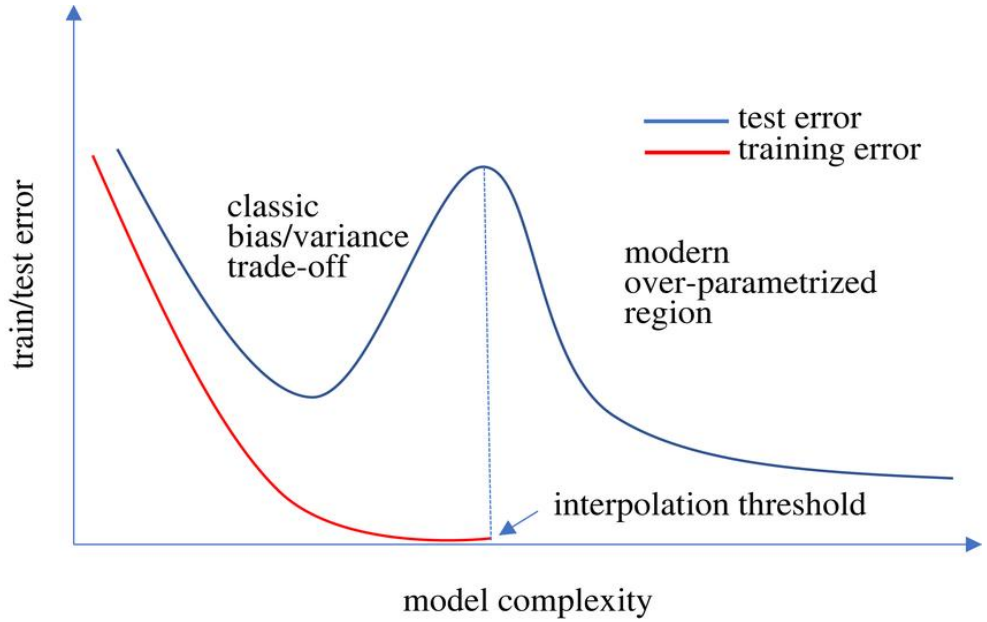
- (Bayesian) Bias is already in-built in prior

No Free-Lunch Theorem

- Training dataset may not contain all possible samples of input
  - How would the unseen data affect the learned statistics?
- No free-lunch theorem: As the unseen data can have different statistics, there can exist a dataset where any model (optimally trained with a training dataset) can perform poorly
  - So, will all models perform equally bad on average?
  - Tradeoff between robustness and accuracy
- Inductive Bias: If some statistics of the entire data is known, use that information in addition to training data to optimize the model – different architectures

Double Descent

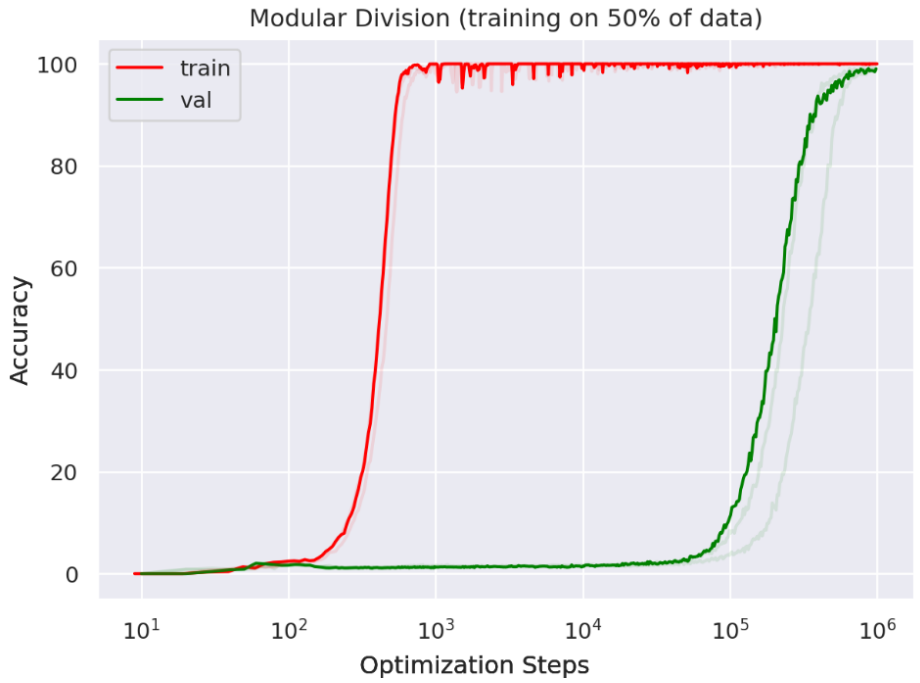
- Overfitting can be overcome
  - With deeper networks
- As the number of parameters (depth) increases, the overfitting is smoothened
  - Smoother interpolations and extrapolations



- Interpolation threshold
  - Where the number of parameters equals the number of data points
  - R. Shaeffer et al, *Double Descent Demystified: Identifying, Interpreting & Ablating the Sources of a Deep Learning Puzzle*
  - Some times more training data can hurt performance!

Grokking

- Generalization of over-parameterized nets by over training
- Amount of over training reduces with decrease in dataset size
  - *Grokking Explained: A Statistical Phenomenon*, 2025
- Weight decay helps
  - Additional term in the loss function to push the weights closer to zero



Assignment

- Generate training 100 samples : $y = x + 0.3x^2 + 0.7x^3 + w$
  - x is uniformly distributed in [-3 to 3]
  - w is Gaussian noise with mean 0 and standard deviation 0.09
- Perform linear regression for model sizes M = 1 to 120 and plot
  - Bias (average of y – f(x)) vs M
  - Variance of y vs M
  - Training error vs M
  - Validation error vs M

Multilayer Perceptron

Feedforward Neural Network

Extending Single-Layer Neural Nets

- For linear data models
  - $y = \mathbf{w}^T\mathbf{x}$ or $\sigma(\mathbf{w}^T\mathbf{x} + b)$
  - Perceptron: $u(\mathbf{w}^T\mathbf{x} + b)$
- In general, $y = f(\mathbf{w}, \mathbf{x})$
  - What are some good parametric functions?
  - Are there a family of such functions that can fit any arbitrary N-dimensional curve?

Universal Function Approximation Theorems

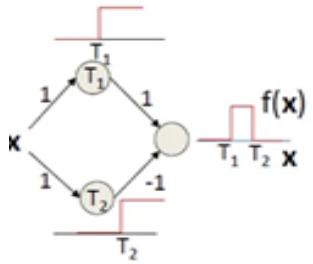
- Stone - Weierstrass (1885)
  - Every continuous function can be approximated by polynomials
- Andrew Barron (1993 - Universal Approximation Bounds for Superpositions)
  - Lipshitz functions can be approximated by infinite-width 1-layer NN
- Cybenko (1989 - Approximation by superpositions of a sigmoidal function)
  - Lipshitz functions can be approximated by infinite-width 1-layer sigmoid NN

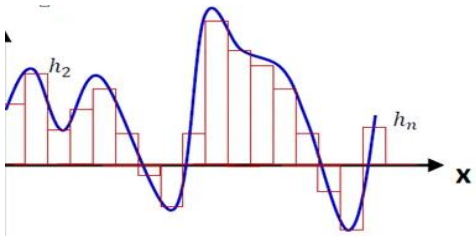
Universal Function Approximation Theorems

- Hornik – Stinchcombe – White (1989 - Multilayer feedforward networks are universal approximators)
  - Lipshitz functions can be approximated by finite-width multi-layer NN
- Lu – Pu – Wang – Hu – Wang (2017 - The expressive power of neural network)
  - Lipshitz functions can be approximated by finite-width multi-layer ReLU NN
- Kolmogorov–Arnold (1957)
  - Lipshitz functions can be approximated by 2-layer NN

Universal Function Approximation

- Polynomials in high dimensions : very high complexity
- Any function f(x) (with some regularity conditions) can be approximated by a superposition of scaled and shifted step functions
  - $f(x) = \Sigma\, a_i\, u(w_i\, x - b_i)$
  - w_i & b_i are the scale and shift factors
  - $u()$ is the activation function

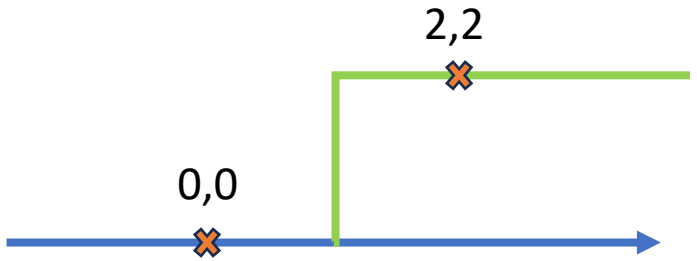




Example

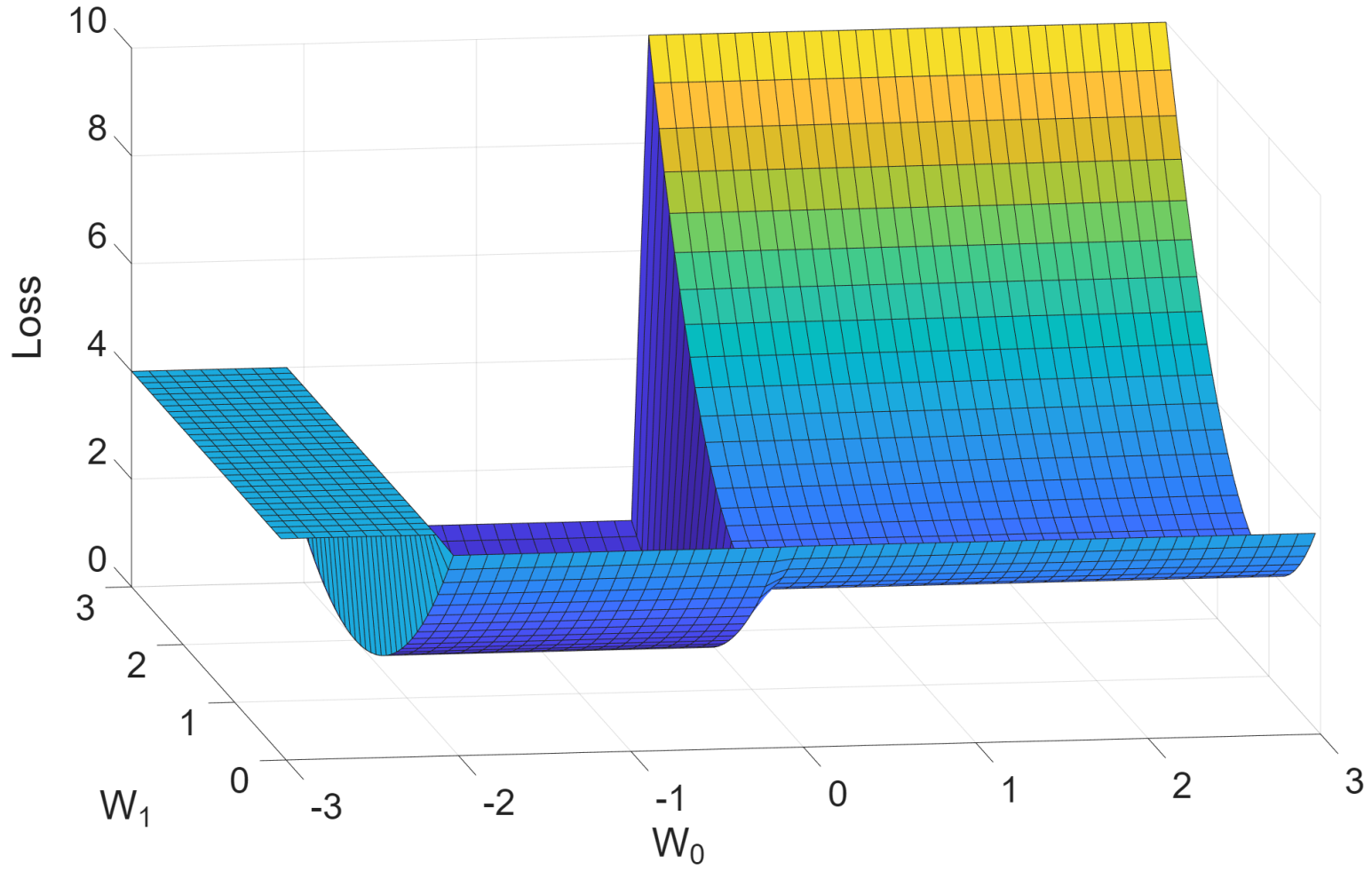
- Fit $w_1 u(x + w_0)$ with MSE loss

$$\text{Loss} = (0 - w_1 u(0 + w_0))^2 + (2 - w_1 u(2 + w_0))^2$$

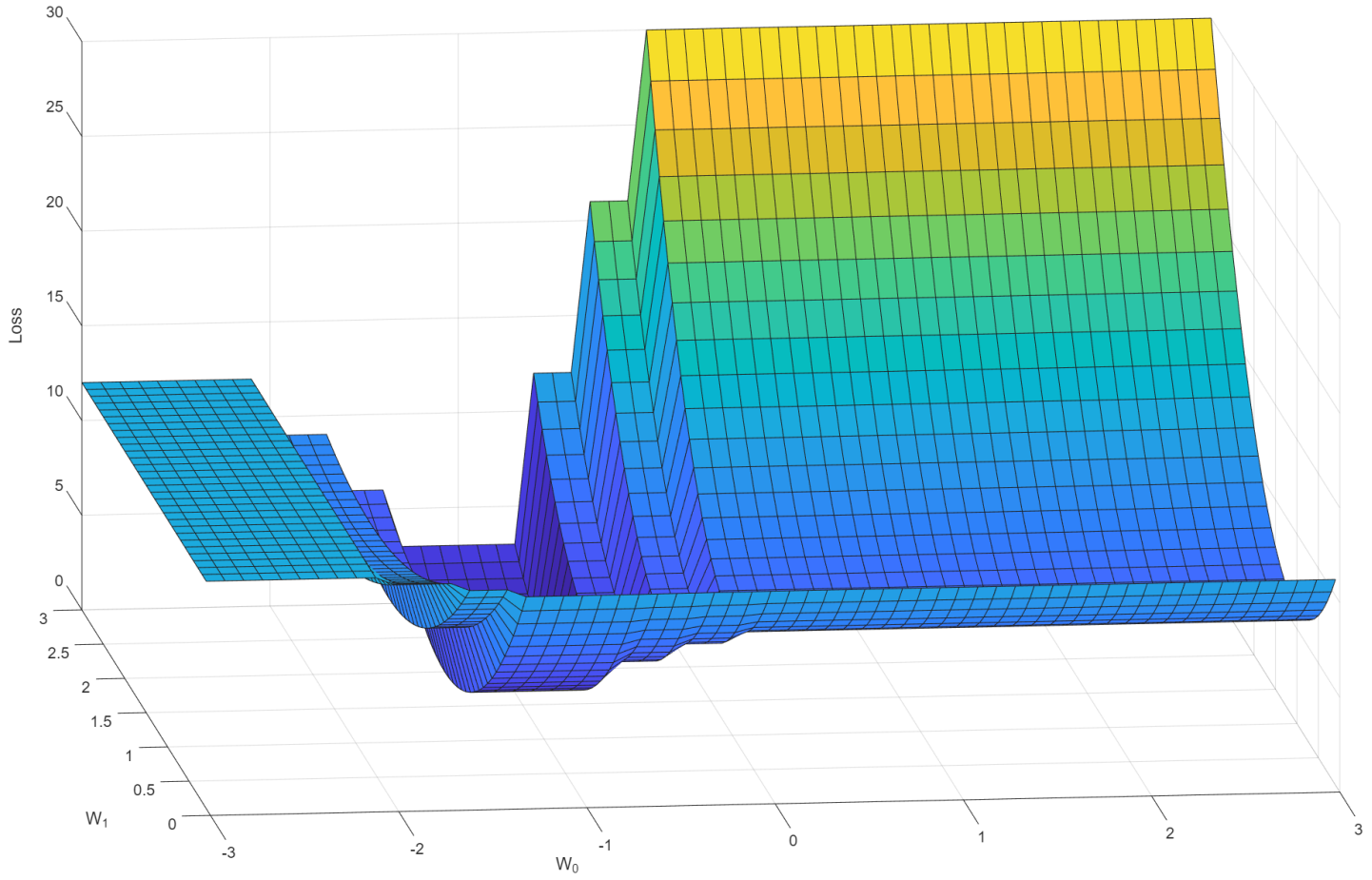


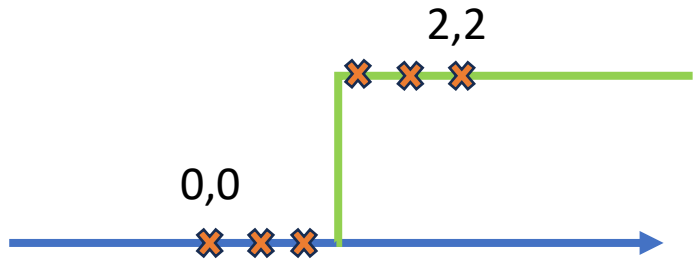
- How does this loss function behave w.r.t. the weights?
- Is it differentiable w.r.t. the weights?

Loss Surface



Loss Surface *Smoothens* with More Data





Challenges

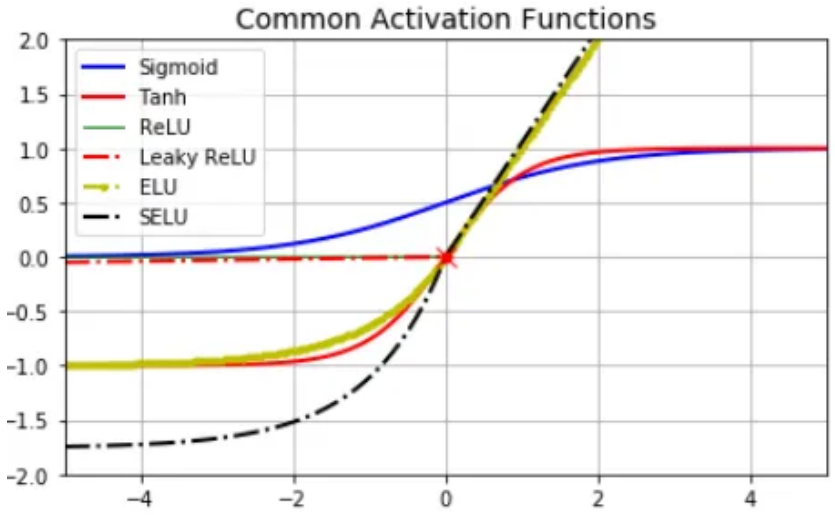
- How to find optimal values?
  - Find roots of derivative expression
- Derivatives should exist
- Roots: Move along the curve to find a flat region
  - Convex surface? Unique solution? Local minima?
  - Initialization point?

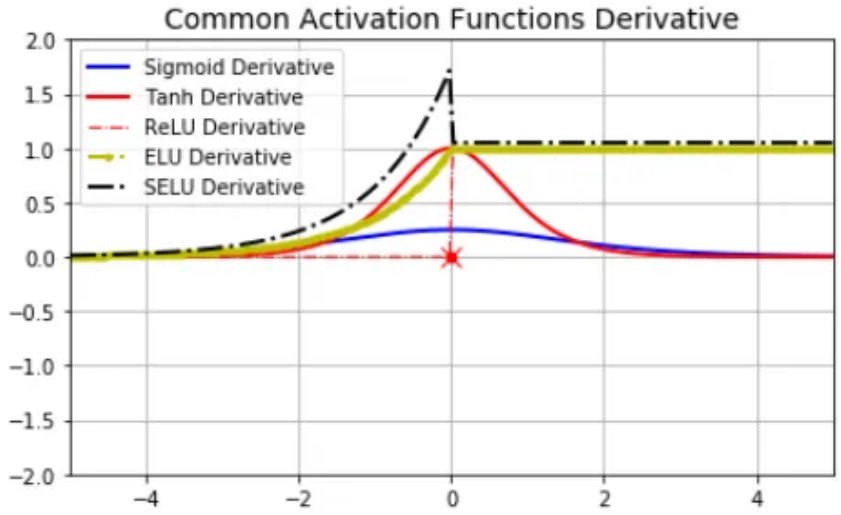
Activation Functions

- Step function has derivative = infinity at zero-input
  - Options: Sigmoid, softmax, tanh (odd symmetry)
- Non-limiting activations:

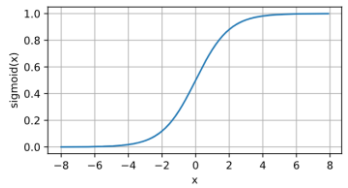
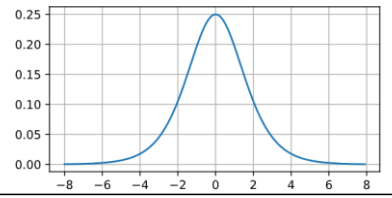
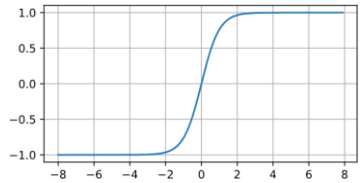
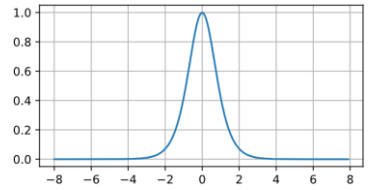
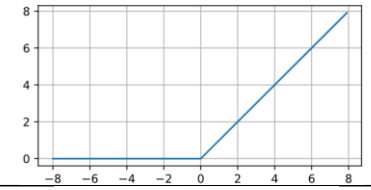
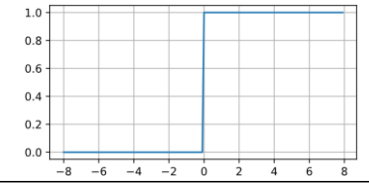
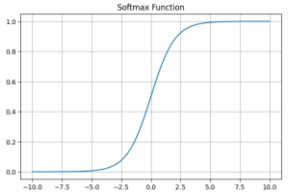
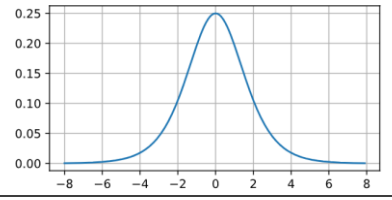
$$\text{selu}(x) = \lambda \begin{cases} x & \text{if } x > 0 \\ \alpha e^x - \alpha & \text{if } x \leqslant 0 \end{cases}$$

  - ReLU (rectified linear unit) & SeLU (scaled exponential linear unit)





Some Common Activation Functions

Function	Expression	Plot	Derivative
Sigmoid	$\frac{1}{1+\exp(-x)}$		
Tanh	$\frac{1-\exp(-2x)}{1+\exp(-2x)}$		
ReLU	$\max(x,0)$		
Softmax	$\frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$		

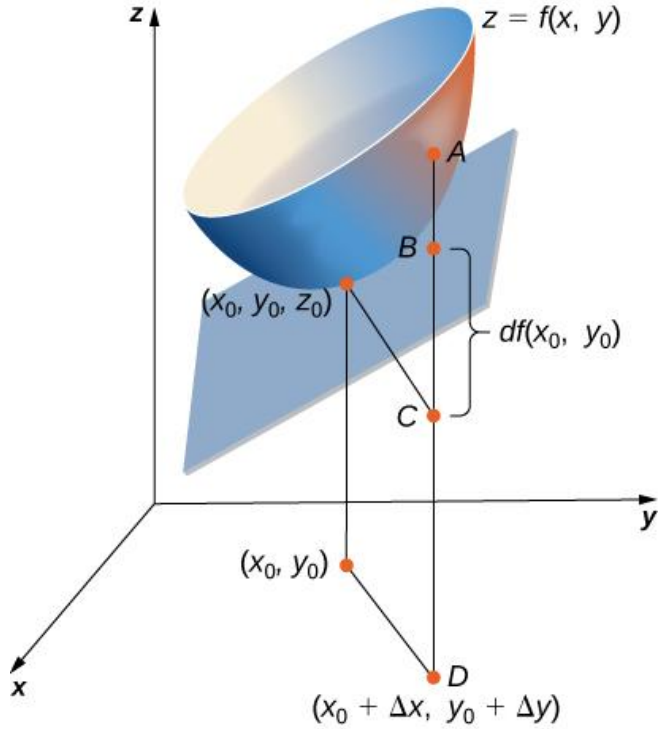
Training Neural Networks

Gradient Descent

- How to find the roots of the derivative of the loss function?
- Taylor approximation in higher dimension
  - Gradient: Linear approximation of a function at a point
  - Hessian: Curvature of a function at a point
- $L(\mathbf{w} + \boldsymbol{\varepsilon}) \approx L(\mathbf{w}) + \nabla L(\mathbf{w})^{\mathrm{T}}(\mathbf{w} + \boldsymbol{\varepsilon} - \mathbf{w})$
  - For $L(\mathbf{w} + \boldsymbol{\varepsilon}) - L(\mathbf{w}) < 0$, pick $\boldsymbol{\varepsilon}$ opposite to $\nabla L(\mathbf{w})$

Gradient Descent

- Gradient descent: $\mathbf{w}_{n+1} = \mathbf{w}_n - \eta \nabla L(\mathbf{w}_n)$
  - $L(\mathbf{w}_0) \geq L(\mathbf{w}_1) \geq \ldots \geq L(\mathbf{w}_n) \geq \ldots$
  - η: step size or learning rate
- How to find gradient w.r.t to all weights?
- What is a good starting $\mathbf{w}_0$?
- What is a good learning rate?



Backpropagation

- For input-label pair (x, y) and cost/loss function C(.)

$$C(y, f^L(W^L f^{L-1}(W^{L-1} \cdots f^2(W^2 f^1(W^1 x)) \cdots)))$$

- The gradient is $\frac{dC}{da^L} \cdot \frac{da^L}{dz^L} \cdot \frac{dz^L}{da^{L-1}} \cdot \frac{da^{L-1}}{dz^{L-1}} \cdot \frac{dz^{L-1}}{da^{L-2}} \cdot \ldots \cdot \frac{da^1}{dz^1} \cdot \frac{\partial z^1}{\partial x}$

$$\nabla_x C = (W^1)^T \cdot (f^1)' \circ \ldots \circ (W^{L-1})^T \cdot (f^{L-1})' \circ (W^L)^T \cdot (f^L)' \circ \nabla_{a^L} C$$

- At each layer, $(f^1)' \circ (W^2)^T \cdot (f^2)' \circ \cdots \circ (W^{L-1})^T \cdot (f^{L-1})' \circ (W^L)^T \cdot (f^L)' \circ \nabla_{a^L} C$
  $(f^2)' \circ \cdots \circ (W^{L-1})^T \cdot (f^{L-1})' \circ (W^L)^T \cdot (f^L)' \circ \nabla_{a^L} C$

  $(f^{L-1})' \circ (W^L)^T \cdot (f^L)' \circ \nabla_{a^L} C$
  $(f^L)' \circ \nabla_{a^L} C,$

Backpropagation with 2-layers

$$\text{Loss} = L\{\overbrace{f_2(\mathbf{W}_2 \overbrace{f_1(\underbrace{\mathbf{W}_1\mathbf{x}}_{=\mathbf{a}_1})}^{=\mathbf{b}_1})}^{=\mathbf{b}_2}; y\}$$

$$\frac{\partial L}{\partial \mathbf{W}_2} = \mathbf{b}_1 \left(f_2' . \frac{\partial L}{\partial \mathbf{b}_2} \right)^T$$

$$\frac{\partial L}{\partial \mathbf{W}_1} = \mathbf{x} \left(f_1' . \mathbf{W}_2^T f_2' . \frac{\partial L}{\partial \mathbf{b}_2} \right)^T$$

Backpropagation

- Compute $\mathbf{g} = \nabla_{\text{output}} L$
- At layer i, (starting from the last layer)
  - $\mathbf{g} = f_i' \mathbf{W}^{\mathrm{T}}_{i+1} \mathbf{g}$
  - Gradient = (current layer input)$\mathbf{g}^{\mathrm{T}}$
  - Repeat till first layer
- How to get $\nabla_{\text{output}} L$?

Initialization

- Can all weights be initialized to 0?
- Why we need random initialization?
- Example: 2 output nodes and 1 input node
  - $y_1 = f(w_1 x + b_1)$
  - $y_2 = f(w_2 x + b_2)$
  - Gradients are similar

Initialization

- Control variance of randomness
  - Xavier: For tanh, sigmoid - $\mathcal{N}(0, 2/n_{l+1}+n_{l-1})$
    - $\mathrm{Var}(\text{output at } l\text{th layer}) = \mathrm{n}_{\mathrm{input}}\, \sigma_{\mathrm{w}}\, \mathrm{Var}(\text{output at } l-1\,\text{th layer})$
    - $\mathrm{Var}(\text{grad at } l\text{th layer}) = \mathrm{n}_{\mathrm{output}}\, \sigma_{\mathrm{w}}\, \mathrm{Var}(\text{grad at } l+1\,\text{th layer})$
  - Kaiming He: For ReLU - $\mathcal{N}(0, 2/n_{l-1})$
    - Almost half of the output of ReLU is zero
  - LeCun: For SELU - $\mathcal{N}(0, 1/n_{l-1})$
  - Random orthogonal matrix

AutoGrad

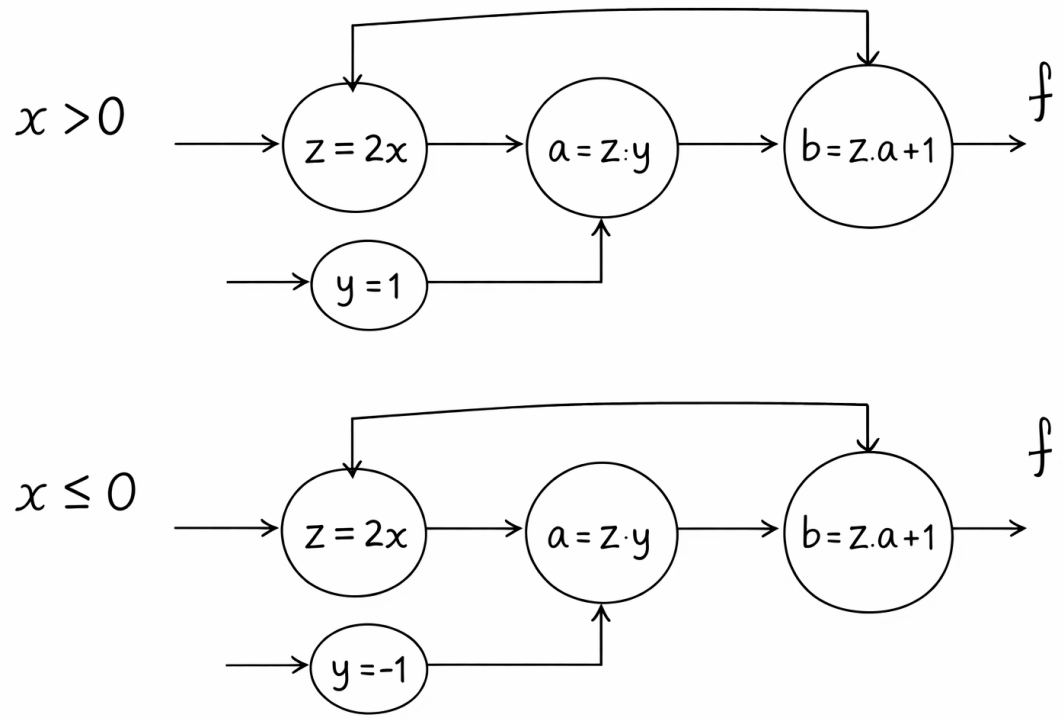
- How to compute gradients of arbitrary models & losses?
- Gradient of a function (in programming) can be computed using **autograd**
  - Exact gradient, not numerical gradient
  - No cycles
  - AG Baydin et al, *Automatic Differentiation in Machine Learning: a Survey*
- Forward pass: compute the output value of all nodes
- Backward pass: compute the gradients using the forward pass values and chain rule of derivatives
- Implemented using computational graphs

Computational Graph

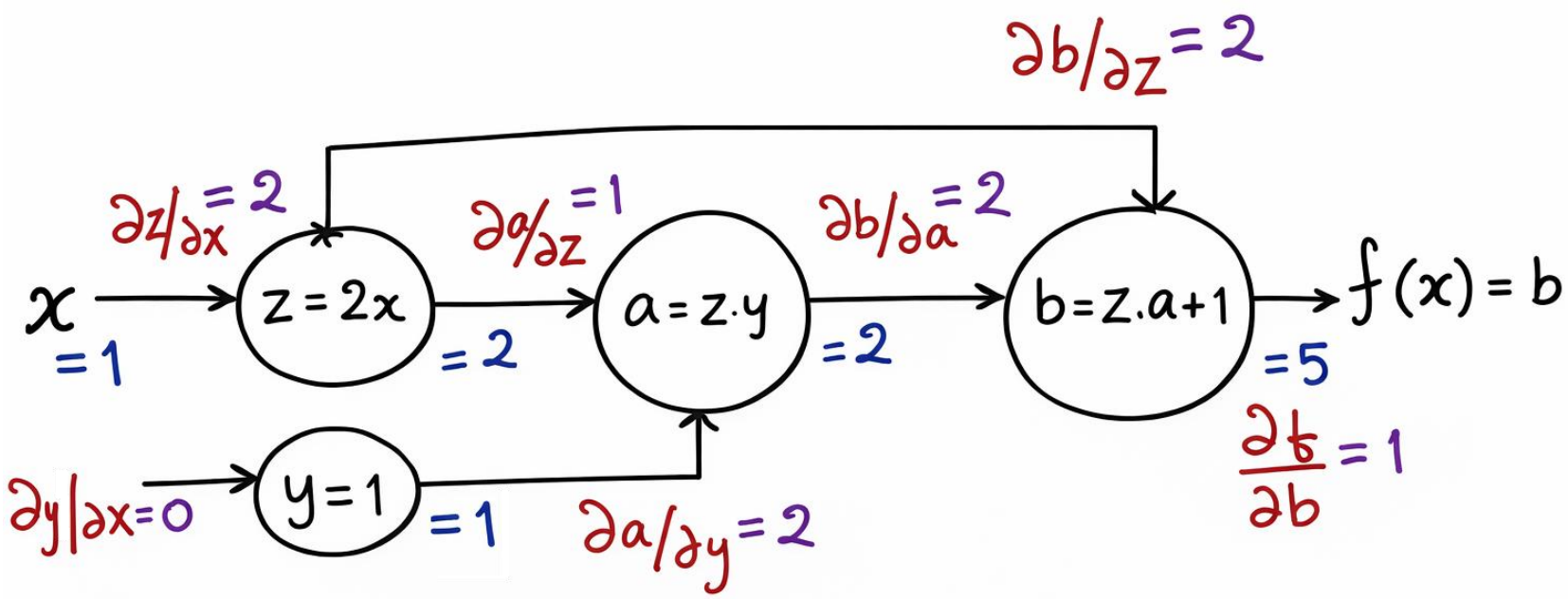
- How to compute gradients of arbitrary models & losses?
- Computational graph: a directed acyclic graph representing all the computations between input and output
  - Dynamic (Loops and conditional statements)
  - Static
  - Output of each (intermediate) computation is a variable or node
    - Function of input variables
  - No cycles (no variable is reused in the graph)

Example Computational Graph

```
def function(x):
        if x>0:
                y = 1
        else:
                y = -1
        z = 2*x
        for i in range(2):
                y = z*y + i
        return y
```



Example Computational Graph



$$\frac{\partial f}{\partial b} = 1 \qquad \frac{\partial b}{\partial a} = z$$

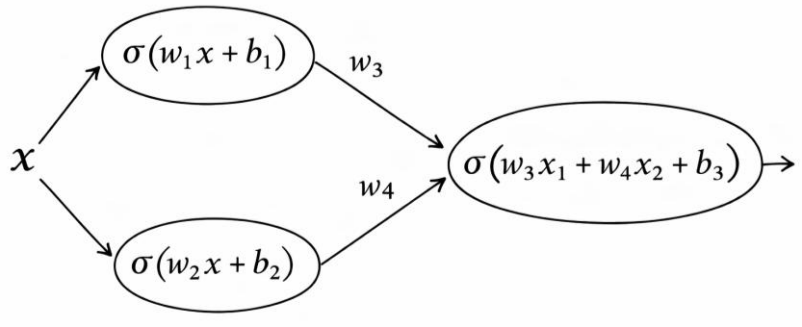
$$\frac{\partial b}{\partial z} = a \qquad \frac{\partial a}{\partial y} = z$$

$$\frac{\partial a}{\partial z} = y \qquad \frac{\partial y}{\partial x} = 0$$

$$\frac{\partial z}{\partial x} = 2$$

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial b}\left[\frac{\partial b}{\partial z}\cdot\frac{\partial z}{\partial x} + \frac{\partial b}{\partial a}\cdot\left(\frac{\partial a}{\partial z}\cdot\frac{\partial z}{\partial x} + \frac{\partial a}{\partial y}\cdot\frac{\partial y}{\partial x}\right)\right] = 2(a + yz)$$

Backpropagation Example



$\frac{\partial z_1}{\partial x} = w_1$ $\quad \frac{\partial v_1}{\partial z_1} = 1$ $\quad \frac{\partial u_1}{\partial v_1} = \sigma'$ $\quad \frac{\partial z_3}{\partial u_1} = w_3$ $\quad \frac{\partial v_3}{\partial z_3} = 1$ $\quad \frac{\partial u_3}{\partial v_3} = \sigma'$

x → $z_1 = x w_1$ → $v_1 = z_1 + b_1$ → $u_1 = \sigma(v_1)$ → $z_3 = w_3 u_1$ → $v_3 = z_3 + z_1 + z_1 + b_3$ → $u_3 = \sigma(v_3)$ → $L = (y - u_3)^2$

x → $z_2 = x w_2$ → $v_2 = z_2 + b_2$ → $u_2 = \sigma(v_2)$ → $z_4 = w_4 u_2$ → $v_3 = z_3 + z_1 + z_1 + b_3$

$\frac{\partial u_2}{\partial v_2} = \sigma'$ $\quad \frac{\partial z_4}{\partial u_2} = w_4$ $\quad \frac{\partial v_3}{\partial z_4} = 1$ $\quad \frac{\partial L}{\partial u_3} = 2(y - u_3)$

$\frac{\partial z_2}{\partial x} = w_2$ $\quad \frac{\partial v_2}{\partial z_2} = 1$

Backpropagation

- Autograd computes gradient through backpropagation
  - Gradients *flow* from one node to another
  - Recursion and matrix operations can have derivatives
    - M. Seeger et al, Auto-Differentiating Linear Algebra, 2019
- Gradients do not flow: vanishing or exploding cases – finite precision
- Autograd fails for
  - Non-differentiable functions
  - Improper computational graph
    - e.g., `if x = 2 then 4 else x * x`, autograd output at 2 = 0 instead of 4
  - Python: array assignment (https://github.com/HIPS/autograd/blob/master/docs/tutorial.md)

Learning NN parameters **w** – Training Phase

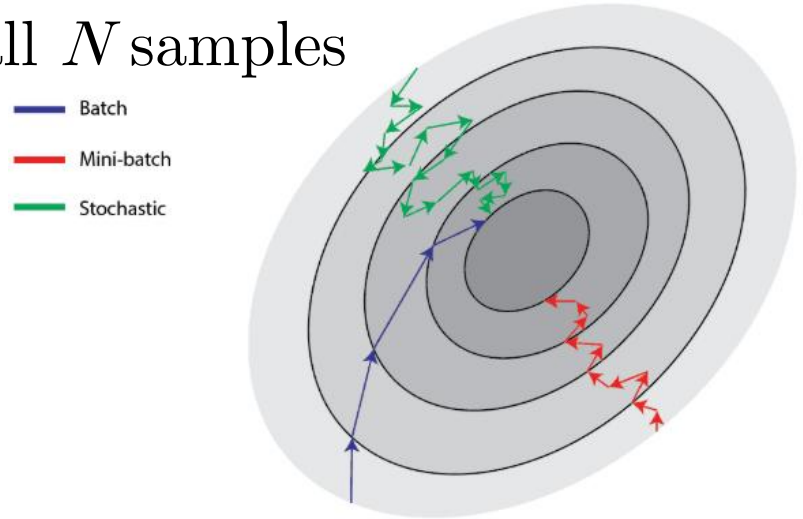
- Initialize w_0 : Xavier, K. Hi, LeCun, etc., and compute loss *-- forward pass*
- For n = 1 to till convergence; Dataset with N samples,
  - Compute/estimate gradient: $\mathbf{g}_n = \sum_i \nabla_{w_{n-1}} L_i / N$
    - For each i, compute $\nabla_{w_{n-1}} L_i$ using backpropagation (autograd) *-- backward pass*
    - SGD, SAG, SAGA
  - Compute learning rate: η
    - AdaGrad, RMSprop, Adam
  - Update parameters: $\mathbf{w}_n = \mathbf{w}_{n-1} - \eta\, \mathbf{g}_n$
    - First order, second order
  - Compute loss: L(label, Model{w_{n+1}, input}) *-- forward pass*
- End

Stochastic Gradient Descent

- What if number of samples is large? Compute N gradients in each step?
- SGD – at each descent step, find gradient for only one random sample:
  - $\mathbf{w}_n = \mathbf{w}_{n-1} - \eta \nabla_w L_j$
  - j is randomly chosen from $\{1,\ldots, N\}$ at every descent step:
    - Estimated gradient is a random (stochastic) sample: $\nabla_w L_j = \nabla_w L + z$ = average (true) gradient + noise
  - Note that, for large N, $\Sigma_i \nabla_w L_i / N \rightarrow E\{\nabla_w L\}$
  - For correlated data samples, $\nabla_w L_j$ could be close to average
  - Uncorrelated samples (noisy gradient) may lead to divergence (variance of z is high)
  - Very slow convergence $O(1/\sqrt{n})$

Stochastic Gradient Descent - Batched

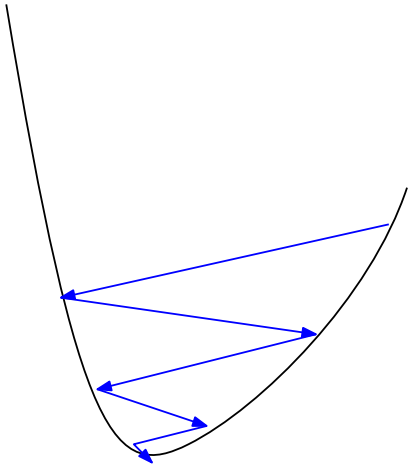
- Improve average gradient estimate through mini-batches
  - SGD with minibatch: $w_n = w_{n-1} - \eta \Sigma_{k=1 \text{ to } M} \nabla_w L_{j_k} / M$
  - j_k is randomly chosen from $\{1,\ldots,N\}$, for $k = 1$ to M, at every descent step
  - Batch size: $M << N$, reduces variance; converges at $O(1/n)$ – theoretical best
- Epoch: Set of descent steps required to go through all N samples
  - GD: 1 epoch = 1 descent step
  - SGD: 1 epoch = N descent steps
  - SGD-minibatch: 1 epoch = N/M descent steps



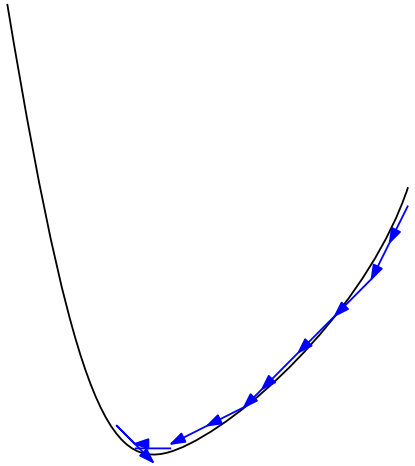
Stochastic Average Gradient

- Can we improve SGD without a mini-batch?
  - SAG: $w_{n+1} = w_n - \eta(\nabla_w L_j^{(n)} - \nabla_w L_j^{(n-1)} + \sum_i \nabla_w L_i^{(n-1)})/N$
  - At step $n = 0$, set $\nabla_w L_i^{(0)} = 0$ and compute average gradient as $\sum_i \nabla_w L_i^{(0)}/N$
  - At step n, j is randomly chosen from $\{1,\ldots,N\}$ and only $\nabla_w L_j^{(n)}$ is evaluated
    - Update average gradient as $(\nabla_w L_j^{(n)} + \sum_{i \neq j} \nabla_w L_i^{(n-1)})/N$
  - For all i, at each step, store gradients $\nabla_w L_i^{(n)}$
  - However, this average gradient is a biased estimate
- SAGA (SAG Accelerated): $w_{n+1} = w_n - \eta(\nabla_w L_j^{(n)} - \nabla_w L_j^{(n-1)} + \sum_i \nabla_w L_i^{(n-1)}/N)$
  - Unbiased estimate of the average gradient; converges at $O(1/n)$
    - Defazio et al "*SAGA: A Fast Incremental Gradient Method With Support for Non-Strongly Convex Composite Objectives*"

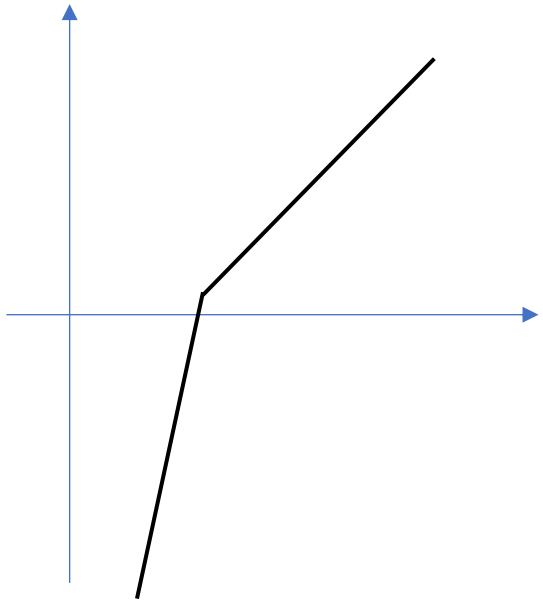
Effect of Learning Rate



Large step size



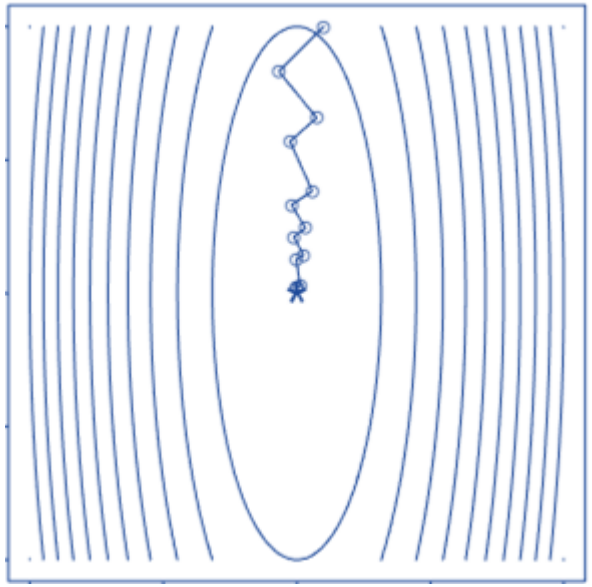
Small step size



Gradient

Optimal Learning rate – Steepest Descent

- Optimal $\eta^* = \text{argmin}_\eta\, L(\mathbf{w}_{n-1} - \eta \nabla \mathbf{w}_{n-1} L) = \text{argmin}_\eta\, L(\mathbf{w}_{n-1} - \eta \mathbf{g}_n)$
  - Not always computable
- At optimal η^*, $\nabla_\eta\, L(\mathbf{w}_{n-1} - \eta \mathbf{g}_n) = 0$
  - That is, $\nabla L(\mathbf{w}_n)^T \mathbf{g}_n = \mathbf{g}_{n+1}{}^T \mathbf{g}_n = 0$
- Consecutive gradients are orthogonal
  - Slow convergence, undoing previous minimizations
  - Fix: differently adapting learning rate or gradient



Adapting Learning Rate

- Linear decay: $\eta_n = (1 - \alpha)\eta_0 + \alpha\eta_t$
  - t: terminal rate instance; $\alpha = \min(1, n/t)$
- AdaGrad: $\eta_n = \alpha/(\varepsilon + \sqrt{\mathbf{h}_n})$; $\mathbf{h}_n = \mathbf{h}_{n-1} + (\Sigma_i \nabla_w L_i^{(n)}/N)^2 = \mathbf{h}_{n-1} + \mathbf{g}_n^2$
  - Change η depending on the magnitude of slope of each coordinate in the gradient
  - Element-wise learning rate, ε is for numerical stability
  - Reduce learning rate when accumulated gradient is large and vice versa
  - (+) Good for sparse gradients
  - (–) decreases learning rate early – vanishing; problem when gradient is noisy
- RMSprop: $\eta_n = \alpha/(\varepsilon + \sqrt{\mathbf{h}_n})$; $\mathbf{h}_n = \beta\mathbf{h}_{n-1} + (1 - \beta)(\Sigma_i \nabla_w L_i^{(n)}/N)^2 = \beta\mathbf{h}_{n-1} + (1 - \beta)\mathbf{g}_n^2$
  - Accumulation is averaged to prevent early decrease in learning rate

Adaptive Moment Estimation

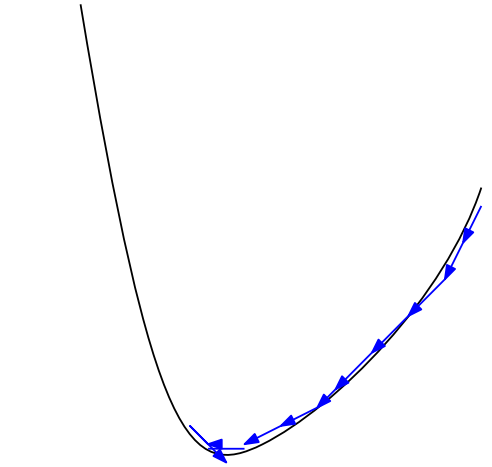
- AdaGrad & RMS prop compute root sum square & RMS of gradients, respectively
  - Results in learning rate decay as $\mathbf{n}$ increases and gradient magnitude accumulates
  - However, the actual direction of gradient is not used to increase or decrease learning rate
- Compute first moment: $\mathbf{m}_n = \beta_1 \mathbf{m}_{n-1} + (1 - \beta_1)\mathbf{g}_n$
  - β_1 is less than 1 (usually very close to 1)
  - $\mathbf{m}_n$ is weighted moving average of gradients – smoother trajectory (thus, skips small local minima)
  - For $\mathbf{m}_0 = 0$ and a constant gradient $\mathbf{g}_n = \mathbf{g}$, $\mathbf{m}_n \rightarrow \beta_1 \mathbf{g}_n$
    - Biased towards zero (as initial point is 0); unbiased moment $= \mathbf{m}_n / (1 - \beta_1^n)$
  - Builds momentum that helps to move out of small wells

Adam Optimizer

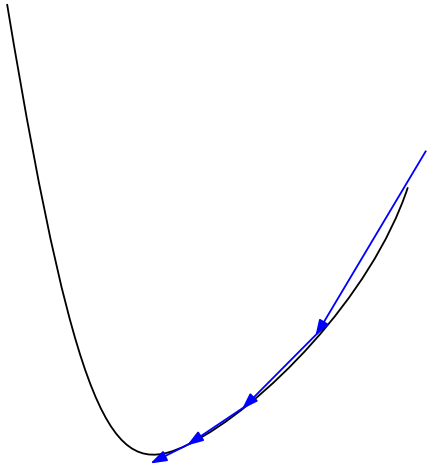
- Large momentum can lead to moving past a significant local minima
- A solution: When the variance of the gradient is high, take smaller steps to not miss local minima and vice versa
- Compute non-central second moment: $\mathbf{v}_n = \beta_2 \mathbf{v}_{n-1} + (1 - \beta_2)\mathbf{g}_n^2$
  - β_2 is less than 1 (usually very close to 1)
  - $\mathbf{v}_n$ is weighted moving average of the (uncentered) variance of the gradients
  - Biased towards zero; unbiased moment $\mathbf{s}_n = \sqrt{(\mathbf{v}_n / (1 - \beta_2^n))}$
  - Divide learning rate by $(\mathbf{s}_n + \boldsymbol{\varepsilon})$
  - Helps to reduce oscillations

Adam Optimizer

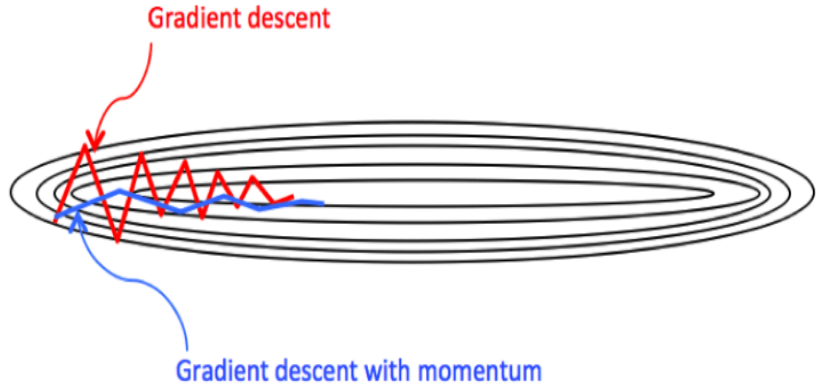
- Adam
  - Compute $\mathbf{m}_n$ and $\mathbf{k}_n = \mathbf{m}_n/(1-\beta_1^n)$
  - Compute $\mathbf{v}_n$ and $\mathbf{s}_n = \sqrt{\left(\mathbf{v}_n/(1-\beta_2^n)\right)}$
  - Update $\mathbf{w}_{n+1} = \mathbf{w}_n - \eta\mathbf{k}_n/(\mathbf{s}_n + \boldsymbol{\varepsilon})$
    - $\boldsymbol{\varepsilon}$ – to avoid division by zero
  - D. P. Kingma et al, *Adam: A Method for Stochastic Optimization*
- Learning rate is adapted coordinate-wise
- Multiple variants of Adam exist in literature



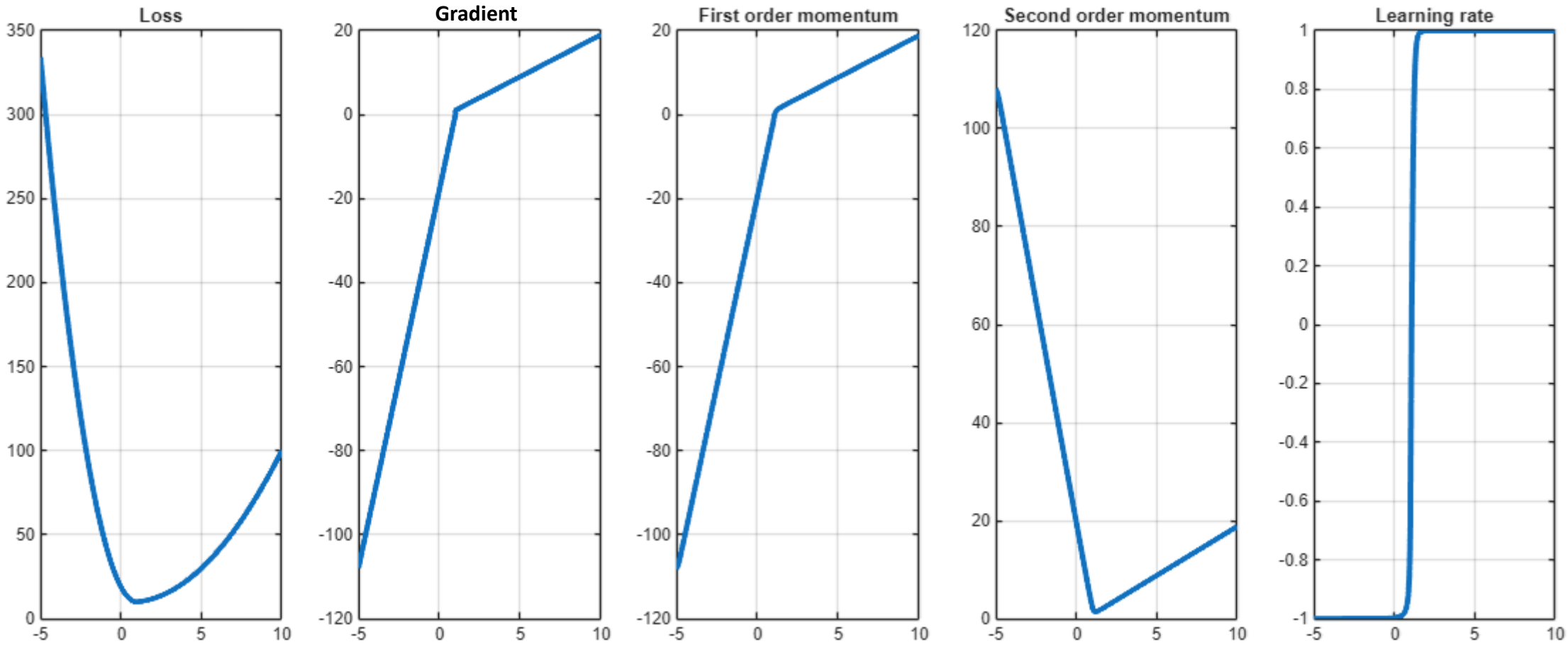
First moment based descent



Adam

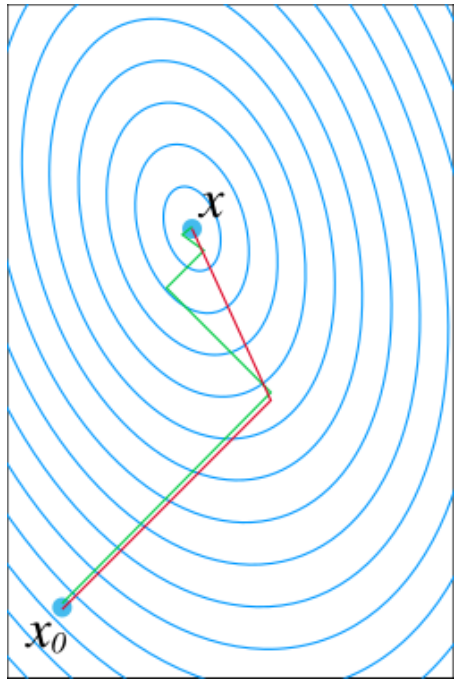


Adam Method


Loss
Gradient
First order momentum
Second order momentum
Learning rate

Conjugate Gradient Descent

- Another option is to change the gradient computation instead of LR
- At step $\mathbf{n}$, if $\mathbf{d}_n$ is the direction along which we are descending
  - Avoid $\mathbf{d}_n^T\mathbf{d}_{n-1} = 0$, by setting $\mathbf{d}_n = \mathbf{g}_n + \beta\mathbf{d}_{n-1}$
  - Such that they are conjugate directions: $\mathbf{d}_n^T\mathbf{H}\mathbf{d}_{n-1} = 0$
    - $\mathbf{H}$ is the Hessian matrix; optimal directions: eigenvectors of $\mathbf{H}$
  - Substituting, we get $\beta = -\mathbf{g}_n^T\mathbf{H}\mathbf{d}_{n-1} / \mathbf{d}_{n-1}^T\mathbf{H}\mathbf{d}_{n-1}$
  - Fletcher-Reeves approximation: $\beta = -\mathbf{g}_n^T\mathbf{g}_{n-1} / \mathbf{g}_{n-1}^T\mathbf{g}_{n-1}$
  - Update step: $\mathbf{w}_n = \mathbf{w}_{n-1} - \eta\,\mathbf{d}_n$



Second-Order: Newton's Method

- $L(\mathbf{w}+\boldsymbol{\varepsilon}) \approx L(\mathbf{w}) + \nabla L(\mathbf{w})^{\mathrm{T}}(\mathbf{w}+\boldsymbol{\varepsilon} - \mathbf{w}) + \boldsymbol{\varepsilon}^{\mathrm{T}}\mathbf{H}\boldsymbol{\varepsilon}/2$
  - Quadratic approximations of loss instead of linear
- Similarly, $\nabla L(\mathbf{w}+\boldsymbol{\varepsilon}) \approx \nabla L(\mathbf{w}) + \mathbf{H}\boldsymbol{\varepsilon}$
  - $\mathbf{H}\boldsymbol{\varepsilon}$ is the change of gradient along the direction $\boldsymbol{\varepsilon}$
  - $\mathbf{v}^{\mathrm{T}}\mathbf{H}\boldsymbol{\varepsilon} = 0 : \mathbf{v}$ is direction orthogonal to the change of gradient
  - If the gradient at the next step has to be zero, then $-\boldsymbol{\varepsilon} \approx \mathbf{H}^{-1}\nabla L(\mathbf{w})$
  - To prevent instability: $-\boldsymbol{\varepsilon} = (\mathbf{H} + c\mathbf{I})^{-1}\nabla L(\mathbf{w})$
  - Update step: $\mathbf{w}_n = \mathbf{w}_{n-1} - \eta\,(\mathbf{H}_{\mathbf{w}_{n-1}} + c\mathbf{I})^{-1}\nabla L(\mathbf{w}_{n-1})$
  - Convergence: $O(1/n^n)$ – superlinear (*Non-asymptotic Global Convergence Analysis of BFGS with the Armijo-Wolfe Line Search, 2024*)

Broyden-Fletcher-Goldfarb-Shanno Optimizer

- Change in weights: $\mathbf{d}_n = \mathbf{w}_n - \mathbf{w}_{n-1}$
- Change in gradient: $\mathbf{h}_n = \mathbf{g}_n - \mathbf{g}_{n-1}$
- Hessian at that location: $\mathbf{H}_n\mathbf{d}_n \approx \mathbf{h}_n$
  - Can we find a matrix $\mathbf{H}_n$ that satisfies this equation?
  - Constraints: symmetric, positive definite, close to $\mathbf{H}_{n-1}$
  - Solve argmin $\|\mathbf{A} - \mathbf{H}_{n-1}\|$ such that $\mathbf{A} = \mathbf{A}^T$ and $\mathbf{A}\mathbf{d}_n = \mathbf{h}_n$
    - Solution: $\mathbf{H}_n^{-1} \approx \mathbf{B}_n = \mathbf{B}_{n-1} + \mathbf{d}_{n-1}^T\mathbf{d}_{n-1}/\mathbf{h}_{n-1}^T\mathbf{d}_{n-1} - \mathbf{B}_{n-1}\,\mathbf{h}_{n-1}^T\mathbf{h}_{n-1}\mathbf{B}_{n-1}/\mathbf{h}_{n-1}^T\mathbf{B}_{n-1}\mathbf{h}_{n-1}$
  - Initialization: $\mathbf{B}_n = \mathbf{0}$; Update step: $\mathbf{w}_n = \mathbf{w}_{n-1} - \eta\mathbf{B}_{n-1}\nabla L(\mathbf{w}_{n-1})$

Nesterov Optimization

- Evaluate gradient/Hessian near the update point
  - Predict the next point and find the gradient there: gradient look-ahead
- Compute $\nabla_{\mathbf{w}_{n-1}+\mu\mathbf{r}_{n-1}} L_i$ instead of $\nabla_{\mathbf{w}_{n-1}} L_i$
  - $\mathbf{r}_n$ is the parameter update term such that $\mathbf{w}_n = \mathbf{w}_{n-1} + \mathbf{r}_n$
  - Convergence rate: $O(1/n^2)$ – first order
- Applicable to all the methods discussed so far and later
  - **NAdam** is popular

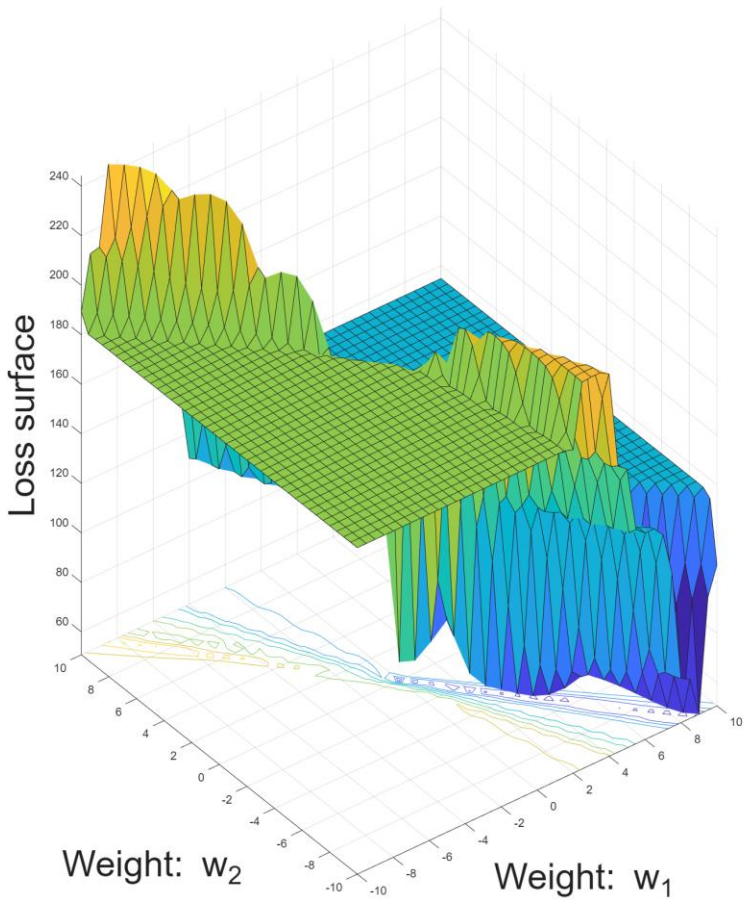
Generalization Performance

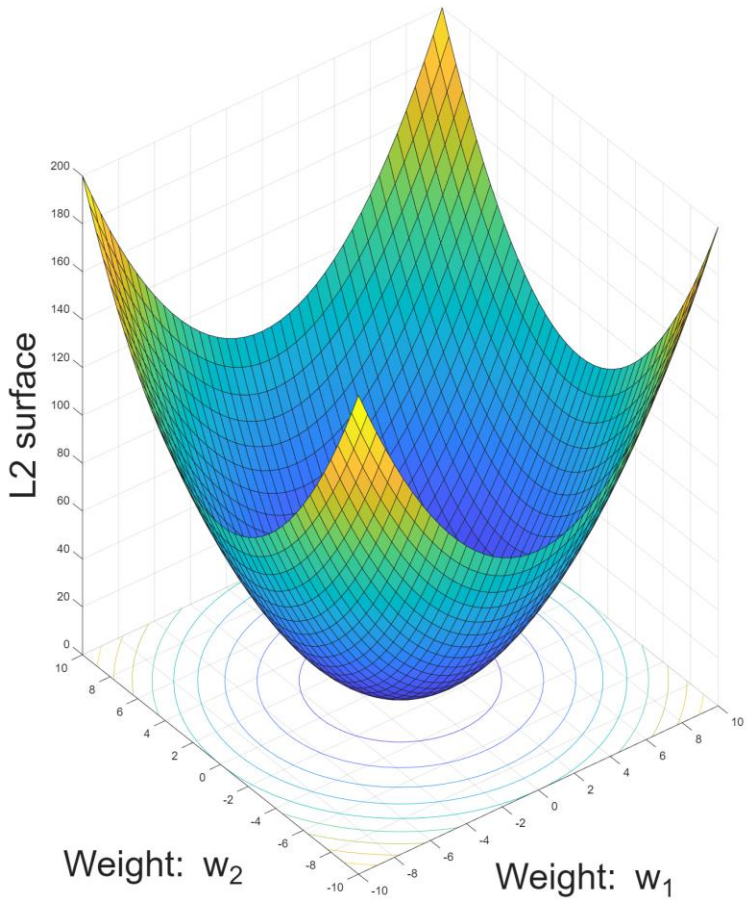
- Does having more layers/neurons always help?
  - Increasing depth/width = increasing number of parameters
  - Recall model selection & bias-variance tradeoff
- Underfitting: training loss is plateauing/increasing
- Overfitting
  - Decreasing training loss, but plateauing/increasing validation loss
- How to get good generalization? Avoid overfitting (memorization)?

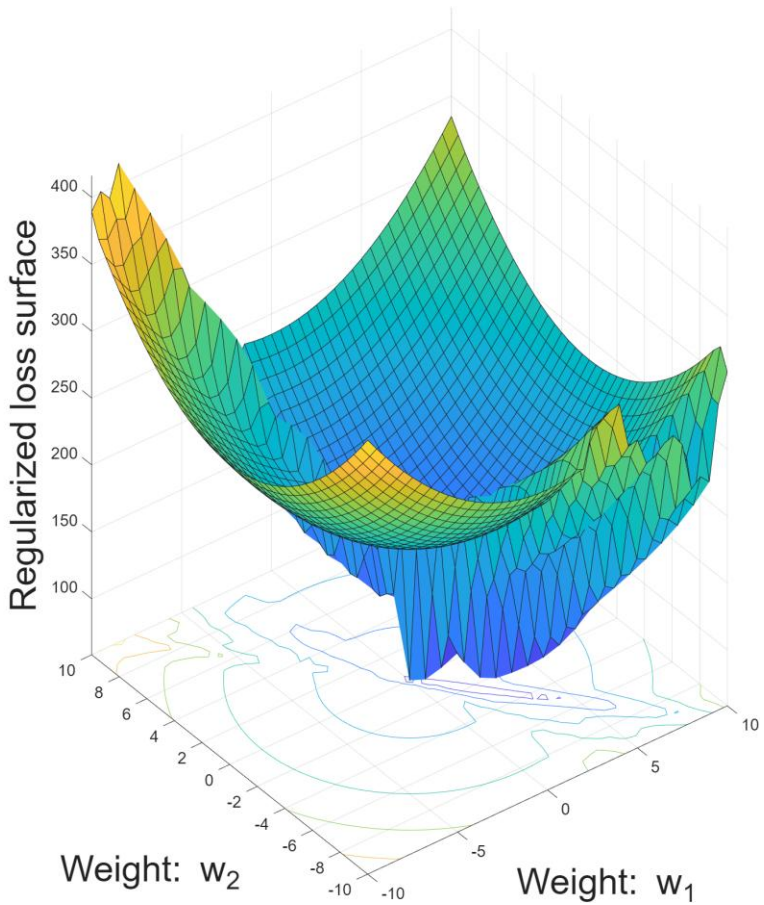
Regularization

- Improve weight/parameter selection through prior information
  - Adding constraints on the parameters
- Weight decay (only for weights, not biases)
  - Loss function: $L(\mathbf{w}) + \lambda \|\mathbf{w}\|^2/2$ -- Smoothens the loss surface
  - L2 or Tikhonov regularization, ridge regression, push weights towards zero
  - Update step: $\mathbf{w}_n = (1 - \eta\lambda)\, \mathbf{w}_{n-1} - \eta \mathbf{g}_{n-1}$
  - L1 regularization as used for sparse weights/parameters
- Parameter sharing: Constrain the weights to be similar to another fixed pretrained neural network: $L(\mathbf{w}) + \lambda \|\mathbf{w} - \mathbf{w}_{\text{fixed}}\|^2$ -- fine tuning

Visualizing Loss Function Smoothening







Data Augmentation

- As we saw earlier, increasing training dataset size improves loss surface & generalization
- Noise injection/perturbation: Add noise to training data
  - Adding Gaussian noise ($\varepsilon \sim \mathcal{N}(0, \sigma^2)$) to input has the same behavior as L2 regularization!
  - Illustration: Loss $= \mathbb{E}(\mathbf{y} - \mathbf{w}^T(\mathbf{x} + \boldsymbol{\varepsilon}))^2 = \mathbb{E}(\mathbf{y} - \mathbf{w}^T\mathbf{x})^2 + \sigma^2 \|\mathbf{w}\|^2$
    - $\mathbb{E}(\mathbf{y} - \mathbf{w}^T(\mathbf{x} + \boldsymbol{\varepsilon}))^2 = \mathbb{E}(\mathbf{y} - \mathbf{w}^T\mathbf{x})^2 + \mathbb{E}(\mathbf{w}^T\boldsymbol{\varepsilon})^2 - \mathbb{E}(2(\mathbf{y} - \mathbf{w}^T\mathbf{x})\ \mathbf{w}^T\boldsymbol{\varepsilon}))$
    - $\mathbb{E}(2(\mathbf{y} - \mathbf{w}^T\mathbf{x})\ \mathbf{w}^T\boldsymbol{\varepsilon})) = \mathbb{E}(\mathbf{a}^T\boldsymbol{\varepsilon}) = 0;$ $\mathbb{E}(\mathbf{w}^T\boldsymbol{\varepsilon})^2 = \mathbf{w}^T\ \mathbb{E}(\boldsymbol{\varepsilon}\boldsymbol{\varepsilon}^T)\mathbf{w} = \sigma^2 \|\mathbf{w}\|^2$

Label smoothing

- Change 0/1 labels as ε and $1 - \varepsilon$ (label noise)
  - One hot encoded label: $\mathbf{y} = [0, \ldots, 1, \ldots 0]^T$
  - If softmax, $\sigma_k(\mathbf{w}^T\mathbf{x}) = 0$, then $\mathbf{w}^T\mathbf{x} \rightarrow -\infty$: exploding gradient and weights
  - Smoothen label by a convex combination of label and uniform distribution
  - Let labels be $\mathbf{z} = (1 - \varepsilon)\mathbf{y} + \varepsilon\mathbf{u} = [\varepsilon/K, \ldots, 1 - \varepsilon + \varepsilon/K, \ldots \varepsilon/K]^T$
  - Cross entropy loss: $H(\mathbf{z}; \sigma_K(\mathbf{w}^T\mathbf{x})) = (1 - \varepsilon)\, H(\mathbf{y}; \sigma_K(\mathbf{w}^T\mathbf{x})) + \varepsilon H(\mathbf{u}; \sigma_K(\mathbf{w}^T\mathbf{x}))$
    - Make outputs to lie inside the simplex instead of on the vertices
    - Uniform distribution has finite distance from any PMF, so loss is never infinity – smoothens loss surface, thereby, decaying weights instead of exploding

Teacher Training and Knowledge Distillation

- Train a smaller (student) network using a larger (teacher) network
  - Knowledge from deep teacher net is distilled to a shallow student net
  - Teacher net is a conventional deep net with CE loss
  - Student's loss: $(1 - \varepsilon)$ H(input; label) + εH(input; teacher's output)
    - Teacher's outputs are soft labels – smoothen; ε depends on temperature parameter
    - Another possible loss: MSE(input; teacher's logits)
    - Better generalization; M. Phuong et al, *Towards Understanding Knowledge Distillation*

Semi-Supervised Learning

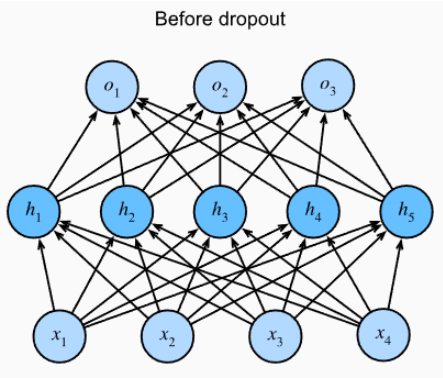
- Similar to data augmentation – generate new data samples instead of adding noise to existing samples
  - Unsupervised learning: Learn data distribution – joint PDF P(input, label) and sample from this distribution to get new samples
  - Examples: Histogram, clustering, kernel density estimation, etc.
  - Autoencoders: A non-linear PCA-like unsupervised learning
    - Encoder: $\mathbf{y} = \mathrm{f}(\mathbf{x})$: high dimension → low dimension; decoder is vice versa
    - Train an autoencoder with given dataset; feed noise to decoder to generate new data

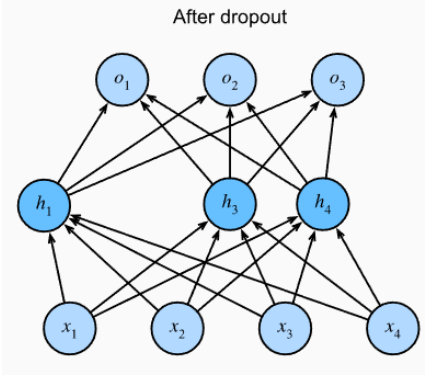
Bagging / Ensemble Learning

- Bootstrapping: Create multiple training datasets by drawing a subset of samples from the original dataset with replacement
- Ensemble: Train multiple neural networks with each of these datasets
- Aggregation
  - Classifier: Maximum voting based inference
  - Regression: Average all the outputs
- Bagging: bootstrap aggregation
  - If error of each network is $e_m(\mathbf{x}) = \mathbb{E}\{f_m(\mathbf{x}) - y(\mathbf{x})\}$,
    then bagging error is $\mathbb{E}\{(f(\mathbf{x}) - y(\mathbf{x}))^2\} = \mathbb{E}\{\Sigma(e_m(x)/M)^2\}$

Dropout

- Another way to implement bagging without high computational complexity
  - Randomly drop a fraction (p) of neurons in hidden layers during training
- Dropout (regularization) – practically effective
  - Multiplicative noise on weights
  - For training with each data point in backpropagation, a mask is used
    - Each iteration trains a different 'subset' of the neural network
    - Training can take longer to converge as gradients are noisy
  - In testing phase, the outgoing weight of a neuron is multiplied with p
    - Output is produced by averaging the outputs of each of these 'subsets'
  - N. Srivastava et al, *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*





Dropout

- Dropout approximates training an exponential number of sub-models simultaneously
- Illustration: loss $= (y - \sum w_i m_i x_i)^2 = y^2 + (\sum w_i m_i x_i)^2 - 2y\sum w_i m_i x_i$
  - m_i is the mask $= 1$ w.p. p and 0 w.p. $1 - p$
  - $\mathbb{E}\{m_i\} = \mathbb{E}\{m_i^2\} = p;\ \mathbb{E}\{m_i m_j\} = p^2$
  - Thus, loss $= (y - p\mathbf{w}^T\mathbf{x})^2 + p(1-p)\sum w_i^2 x_i^2$
  - If $\mathbb{E}\{x_i^2\} = 1$, then loss $= (y - p\mathbf{w}^T\mathbf{x})^2 + p(1-p)\|\mathbf{w}\|^2$

Early Stopping

- Typically, during training: simple patterns are learned first, followed by complicated patterns (possibly noisy - overfitting)
- Stop training if validation loss did not improve beyond p epochs
  - Patience parameter: p
- Produces similar effect as L2 regularization

- L. Prechelt, *Early Stopping – but when?*

Exploding/Vanishing Gradients

- Deeper networks have more expressivity due to more parameters
  - Overfitting due to more parameters is avoided through regularization
- Many layers → product of many gradients
  - Exploding or vanishing
  - Activation function with smaller gradients (sigmoid) can vanish
  - Poor choice of initial point can lead to explosion
- All of the regularization methods, by some means, attempts to keep the gradients stable

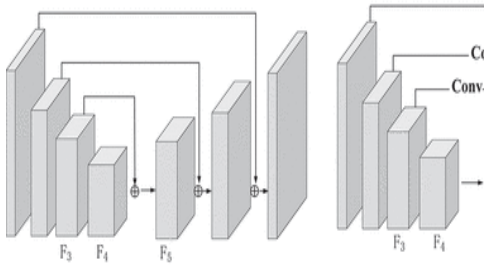
Skip Connection

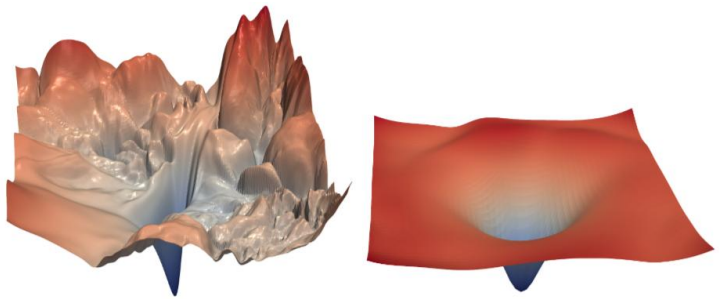
- Avoid vanishing/exploding gradient by skipping between layers

  - E.g., $g_3(\mathbf{W}_3\{g_2(\mathbf{W}_2 g_1(\mathbf{W}_1 x + \mathbf{b}_1) + \mathbf{b}_2) + g_1(\mathbf{W}_1 x + \mathbf{b}_1)\} + \mathbf{b}_3)$

- Randomly choose to add the output of a layer to that of another layer ahead
  - Short skip (between layers of same size) - *ResNet*
  - Long skip

- Skip connection via concatenation
  - Increase output size - *DenseNet*





(a) without skip connections (b) with skip connections

- H. Li et al, *Visualizing the Loss Landscape of Neural Nets*

Hyperparameter Tuning

- Tuning step 1: Choose a search set for: depth, width, activation functions, GD & regularization parameters
- Tuning step 2: Train the network
  - Step 0: Pick a depth, width and activation function from the search set
  - Step 1: Initialize weights and biases (with random values)
  - Step 2: Split data to training + validation
  - Step 3: Do gradient descent (e.g., SGD with Adam) over the loss function by backpropagating (include early stop, dropouts and skip connections if needed)
  - Step 4: Stop when gradient descent converges and compute the training loss and validation loss
  - Step 5: Go to step 1 & repeat until validation loss converges
- Tuning step 3: Go to step 2 till the search set is exhausted
- Tuning step 4: Pick the model with the least training and validation losses